

2

复习

理论课程



福州大学
FUZHOU UNIVERSITY

6

C控制语句： 分支和跳转

理论课程



福州大学
FUZHOU UNIVERSITY

if 语句：格式

- 格式

```
if (<条件表达式>
    <条件为真的语句（体）>
```

- if语句可以相互串联

```
if (<条件表达式>
    <条件为真的语句（体）>
else
    <条件为假的语句（体）>
```

```
if (<条件1表达式>
    <条件1为真的语句（体）>
else if (<条件2表达式>
    <条件2为真的语句（体）>
else
    <条件1和条件2为假的语句（体）>
```

单字符输入 getchar()

getchar

功能	每次从标准输入流读取一个字符并返回。	
格式	<code>int getchar(void);</code>	
参数	无	
返回值	成功	所获取的字符值。
	失败	如果输入流关闭，返回-1（宏EOF）。
头文件	<code>stdio.h</code>	
说明	无。	

单字符输出 putchar()

putchar

功能	每次向标准输入流写入一个字符，并将该字符返回。	
格式	<code>int putchar(int ch);</code>	
参数	<i>ch</i>	以无符号 <code>char</code> 强制转换为 <code>int</code> 的形式返回写入的字符。
返回值	成功	所获取的字符值。
	失败	如果输入流关闭，返回-1（宏EOF）。
头文件	<code>stdio.h</code>	
说明	1. 字符 <i>ch</i> 将转换为无符号字符型。	

```
// cypher2.c -- alters input, preserving non-letters
#include <stdio.h>
#include <ctype.h>
int main(void) {
    char ch;
    while ((ch = getchar()) != '\n')
    {
        if (isalpha(ch))
            putchar(ch + 1);
        else
            putchar(ch);
    }
    putchar(ch);

    return 0;
}
```

// for isalpha()

有经验的程序员将少数表达式嵌套合并在一起节省篇幅，但不应过长或过于复杂。

isalpha(ch)应包含ctype.h头文件，相当于
`ch<='z' && ch>='a' || ch<='Z' && ch>='A'`

This is a cat. That is a dog.↵
 Uijt jt b dbu. Uibu jt b eph.

多分支：switch-case语句

- 语法

把case看成标签

- switch后只允许**整型表达式**
- case后只允许**确定值的整型表达式**
 - 如：a-a, 1.....等

- 功能

- 根据表达式值转到相应case标签
 - 类似goto
- 除非遇到break语句，一直运行到switch语句结束
- 如果没有找到合适的值，则转至default语句

```
switch (<表达式>)  
{  
    case <值1>:  
        <语句（体） 1>;  
    case <值2>:  
        <语句（体） 2>;  
    case <值3>:  
        <语句（体） 3>;  
    default:  
        <语句（体） 0>;  
}
```

多分支：switch-case语句

```
switch (<1 表达式>
{
case <值1>:
    <语句 (体) 1>;
    break;
case <值2>:
    <语句 (体) 2>;
    break;
case <值3>:
    <语句 (体) 3>;
    break;
default:
    <语句 (体) 0>;
}
<4 分支后语句 (体) >;
```

```
switch (<1 表达式>
{
case <值1>:
    <语句 (体) 1>;
case <值2>:
    <2 语句 (体) 2>;
case <值3>:
    <3 语句 (体) 3>;
default:
    <4 语句 (体) 0>;
}
<5 分支后语句 (体) >;
```


题目

1-2 分数 1 查看题目详情

if(a=5) 是允许的。

☐ T

☐ F

T。

if(a = 5)是合法的，因为 C 语言中赋值表达式有返回值（这里是 5），非零即为真，所以条件成立。但这是赋值而非比较，通常是个编程错误。

1-12 分数 1 查看题目详情

如果变量已经正确定义，则执行以下程序段后，x的值不变。

```
if (x = 20) {  
    y = 1;  
} else {  
    y = 0;  
}
```

☐ T

☐ F

F。

if(x = 20)中的 = 是赋值运算符，会将 x 的值改为 20，因此 x 的值发生了变化。虽然原题说“x的值不变”，但实际执行后 x 被赋值为 20，所以 答案是 F。

题目

2-1 分数 1 查看题目详情

判断 i 和 j 至少有一个值为非0的表达式是___。

- ☐ A. $i!=0 \ \&\& \ j!=0$
- ☐ B. $i+j!=0$
- ☐ C. $i \parallel j$
- ☐ D. $i\&\&j$

2-2 分数 1 查看题目详情

表达式_不能用于判断“ x 和 y 不同时为零”。

- ☐ A. $x!=0 \parallel y!=0$
- ☐ B. $x \parallel y$
- ☐ C. $!(x==0 \parallel y==0)$
- ☐ D. $!(x==0 \ \&\& \ y==0)$

C。

判断 i 和 j 至少有一个为非0，即“ i 不为0 或 j 不为0”，对应逻辑或运算。选项 C: $i \parallel j$ 正确表达了这一含义，只要 i 或 j 中有一个非0，表达式就为真。

其他选项：A 要求两者都非0；B 是和不为0（可能两个负数相加也不为0但都不为0）；D 是与，要求两者都非0。

C

“ x 和 y 不同时为零”意味着至少有一个非零，等价于 $x \neq 0 \parallel y \neq 0$ 或 $x \parallel y$ 。选项C $!(x==0 \parallel y==0)$ 等价于 $x!=0 \ \&\& \ y!=0$ ，要求两者都非零，条件过强，不符合题意，因此不能使用。

题目

2-11 分数 1 查看题目详情

下列程序段的输出结果是 ()。

```
int main(void)
{
    int  a = 2, b = -1, c = 2;

    if(a < b)
        if(b < 0)
            c = 0;
    else c++;
    printf("%d\n",c);

    return 0;
}
```

C。

else在 C 语言中总是和最近且尚未匹配的 **if** 配对，所以这里 **else c++**; 实际上对应的是 **if(b < 0)**，两者都在 **if(a < b)** 的内部；但由于 **a = 2, b = -1**，条件 **a < b** 为假，整个内层 **if(b < 0) ... else c++**; 都不会执行，**c** 始终保持初值 2，因此输出为 2。

- ☐ A. 0
- ☐ B. 1
- ☐ C. 2
- ☐ D. 3

题目

2-18 分数 1 查看题目详情

当字符型变量ch 取何值（或何范围）时输出“YES”

```
char ch;  
scanf("%c",&ch);  
if(ch<='A'&&ch>='Z')  
printf("NO");  
else printf("YES");
```

- ☐ A. 大写字母A~Z
- ☐ B. 任何字符
- ☐ C. 除大写字母以外
- ☐ D. 大写字母B~Y

B。

'A'的 ASCII 值比 'Z' 小，所以不可能有字符同时满足 `ch <= 'A' && ch >= 'Z'`，这个条件永远为假，if 永远不执行，程序总是走 else 输出 "YES"，即对任意字符都会输出 YES。

7

C控制语句： 循环

理论课程



福州大学
FUZHOU UNIVERSITY

三种类型的循环

- 三种类型的循环

- do-while 循环的循环体至少执行一次

选用何种循环应根据实际而定，使得代码简单

```
[初始状态;]
while (<循环条件表达式>)
{
    [循环语句 (含状态的改变)]
}
```

```
[初始状态;]
do
{
    [循环语句 (含状态的改变)]
}
while (<循环条件表达式>);
```

```
for ([初始状态]; [循环条件表达式]; [循环状态的改变])
{
    [循环语句 (可包含状态的改变)]
}
```

不确定循环和计数循环

• 两种循环

循环	特点	示例代码
不确定循环	不能预先知道循环了多少次	<pre>for (int i = 0; str[i] != '\0'; i++) { str[i] = str[i] - 'a' + 'A'; }</pre>
计数循环	存在计数器，初始化，判断条件，更新计数	<pre>for (int i = 0; i < N; i++) { str[i] = str[i] - 'a' + 'A'; }</pre>

```
for (初始状态; 循环表达式; 状态改变) {  
    循环体;  
}
```

当初始状态、循环条件、状态改变可以在40字符内写完时，用for语句书写，否则用while。

循环的中止 (break)

- 循环运行到break语句，跳出语句体。

```
① 始状态;]
while (<② 件表达式>)
{
    ③ 环语句]
    if (<④ 出条件>)
        ⑤ break;
    [循环语句]
}
⑥ 环体之外的语句]
```

```
for (① 始状态]; ② 件表达式]; [状态改变])
{
    [③ 环语句 (可包含状态的改变)]
    if (<④ 出条件>)
        ⑤ break;
    [循环语句 (含状态的改变)]
}
⑥ 环体之外的语句]
```

说明: ●表示条件满足和语句成功执行, ●表示条件不满足。

循环的继续 (continue)

- `continue`语句中断本层循环，跳到本次循环结束
 - `while`循环跳入判断表达式；`for`循环跳到状态改变语句

```
1 始状态;]
while (2 件6 达10 >)
{
    3 环7 句]
    if (<4 出8 件>)
        5 continue;
    9 循环语句]
}
11 环体之外的语句]
```

```
for (1 始状态; 2 件7 达12 >; 6 态11 变)]
{
    3 不8 句]
    if (<4 出9 件>)
        5 continue;
    10 循环语句 (含状态的改变) ]
}
13 环体之外的语句]
```

说明： ● 表示条件满足和语句成功执行， ● 表示条件不满足。

嵌套循环 (nest loop)

- 循环内部可以嵌套循环

```
for (row = 0; row < ROWS; row++) {  
    for (ch = 'A'; ch < ('A' + CHARS); ch++)  
        printf("%c", ch);  
    printf("\n");  
}
```

- 循环中的中止和继续语句只能退出当前循环

```
for (i = 0; i < 10; i++) {  
    for (j = 0; j < 10; j++) {  
        if ((i * j) % 2 == 0)  
            continue;  
        if ((i * j) % 13 == 0)  
            break;  
        printf("(%d, %d) ", i, j);  
    }  
    printf("\n");  
}
```

无条件跳转（goto）

goto

功能	无条件跳转	
格式	goto tag;	
参数	tag	同一个函数内的标签。
说明	1. 无条件跳转的标签仅限于同一个函数内。 2. 尽量避免使用回向goto和长距离goto，即尽量避免跳转回之前语句的标签，避免跳转到太远的标签。 3. 结构化编程建议使用其他语句替代goto。	

使用场景

- 代码异常处理
- 一次性跳出多层循环

```
while (exp1) {  
    while (exp2) {  
        if (exp3) {  
            goto out;  
        }  
    }  
}  
out: printf("Exit!\n");
```

```
int mystrlen(char* str) {  
    int ret=0;  
    if (str == NULL) {  
        ret = -1;  
        goto error_tag;  
    }  
    if (*str == 0) {  
        ret = 0;  
        goto error_tag;  
    }  
    while (*str != 0) {  
        ret++;  
        str++;  
    }  
    return ret;  
error_tag:  
    fprintf(stderr, "%s", "Empty.\n");  
    return 0;  
}
```

题目

2-4 分数 2 查看题目详情

下面的这个循环的循环次数是（ ）。

```
for(int i=0,j=10; i=j=10; i++,j--)
```

- ☐ A. 语法错误，不能执行
- ☐ B. 无限次
- ☐ C. 10
- ☐ D. 1

B。

$i=j=10$ 是赋值表达式，不是比较：每次判断条件时先把 j 赋值为 10，再把 10 赋给 i ，整个表达式的值始终是 10（非 0，即条件恒真），而且每轮又把 i 、 j 重置为 10，所以循环永远不会结束，是无限次循环。

题目

2-20 分数 2 查看题目详情

假设变量 `x` 数据类型为 `int`，且初始值为2，则执行下面代码段后

```
while(x--);  
printf("%d\n",x);
```

程序的输出结果是:()

- ☐ A. 2
1
0
- ☐ B. -1
- ☐ C. 1
0
- ☐ D. 死循环

B。

`while(x--);`这一行末尾有分号，表示这是一个“空循环体”，循环只负责不断执行 `x--`，直到 `x` 自减到 -1 时条件为假退出，真正的语句块 `{ printf("%d\n", x); }` 是在循环结束后才执行一次，所以最后输出的是已经减到 -1 的 `x`。

8

字符输入输出 和输入验证

理论课程



福州大学
FUZHOU UNIVERSITY

输入输出缓冲

- 缓冲（buffer）

- 缓冲区是临时存储数据的一段内存区域。
- 缓冲技术用于协调速度相差大的设备间传送数据。
 - 如：CPU和键盘的速度相差很大

- 输入输出缓冲区

- 输入缓冲区（以键盘为例）
 - 键盘将字符送入缓冲区，程序从缓冲区中读取数据
- 输出缓冲区（以屏幕为例）
 - 程序将字符送入缓冲区，缓冲区清空时显示在屏幕上


```
/* echo.c -- repeats input */  
#include <stdio.h>  
int main(void)  
{  
    char ch;  
  
    while ((ch = getchar()) != '#')  
        putchar(ch);  
  
    return 0;  
}
```

```
This is my brother.  
This is my brother.  
$$@$#  
$
```

```

/* showchar2.c -- prints characters in rows and columns */
#include <stdio.h>
void display(char cr, int lines, int width);
int main(void) {
    int ch;           /* character to be printed */
    int rows, cols;   /* number of rows and columns */
    printf("Enter a character and two integers:\n");
    while ((ch = getchar()) != '\n') {
        if (scanf("%d %d", &rows, &cols) != 2)
            break;
        display(ch, rows, cols);
        while (getchar() != '\n')
            continue;
        printf("Enter another character and two integers;\n");
        printf("Enter a newline to quit.\n");
    }
    printf("Bye.\n");
    return 0;
}

```

```

void display(char cr, int lines, int width) {
    int row, col;
    for (row = 1; row <= lines; row++) {
        for (col = 1; col <= width; col++)
            putchar(cr);
        putchar('\n'); /* end line and start a new one */
    }
}

```

Enter a character and two integers:

c 1 3

ccc

Enter another character and two integers;

Enter a newline to quit.

* 2 3

Enter another character and two integers;

Enter a newline to quit.

Bye.

9

数组

理论课程



福州大学
FUZHOU UNIVERSITY

内存中的数据

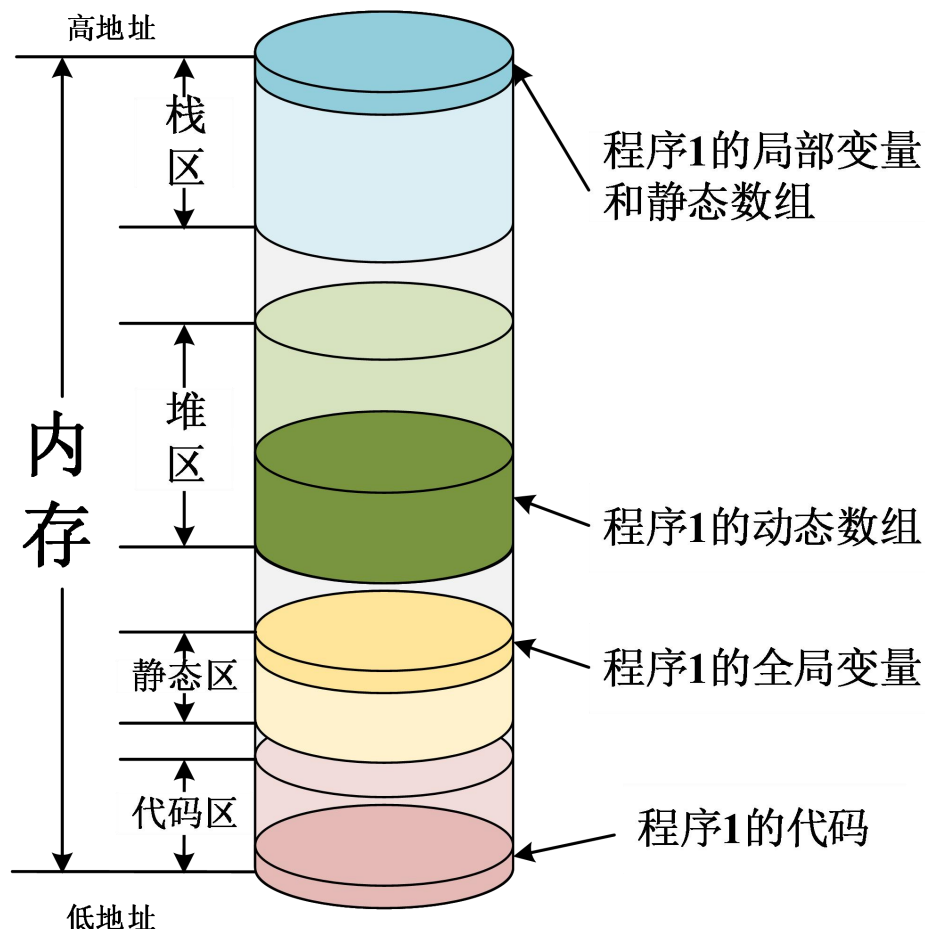
- 可执行程序指令和变量在运行时存储于内存中

- 内存四区

- 栈区 (Stack)
- 堆区 (Heap)
- 静态区 (Static)
- 代码区 (Code)

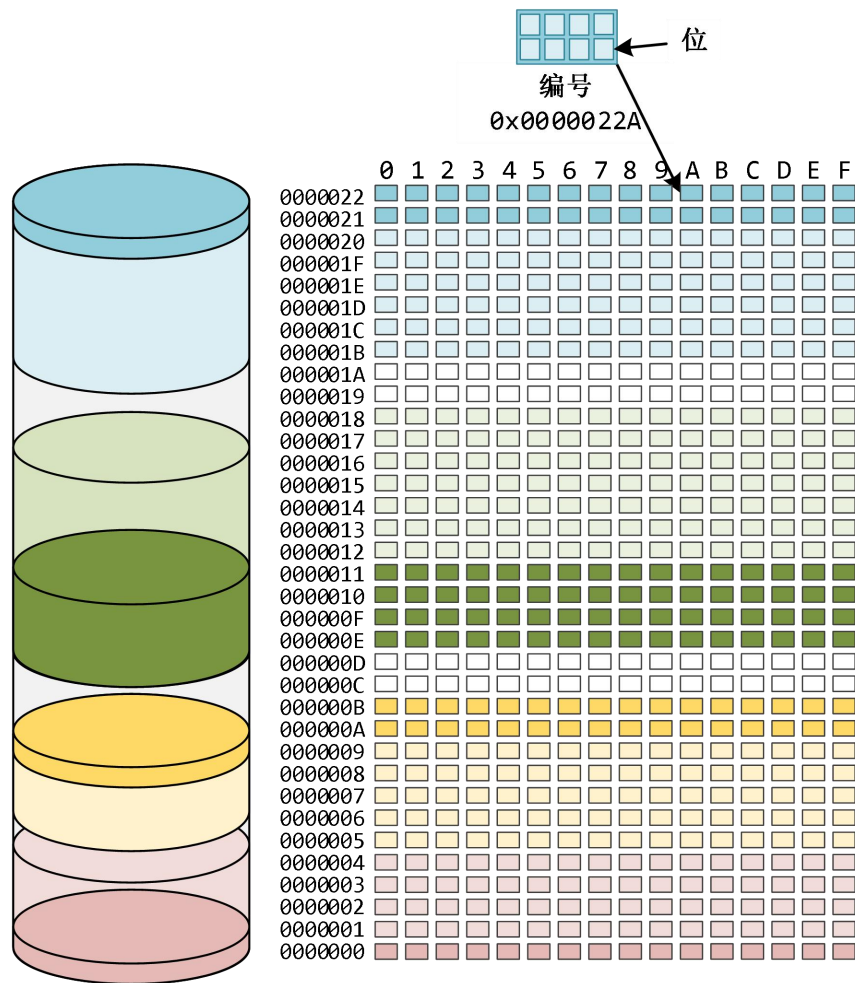
- 内存分配

- 内存的分配不是连续的



内存中的数据

- 位：处理数据的基本单位
 - 两种状态，值为 0 或 1
- 字节：存储的基本单位
 - 1字节折合8位，0—255的整数
 - 至少 $2^3=8$ 位足以表示英文字母
- 内存地址：对字节的编号
 - 最小值：0
 - 最大值：受各因素容量限制



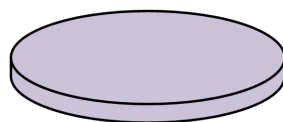
声明变量时的内存分配

- 执行声明语句时

```
int a;
```

- 内存栈区中开辟一个空间
- 从而变量具有

- 内存地址（偏移量）
- 占内存的长度
- 值（取决于长度和位的组织格式）



	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
000001E																
000001D																

000001EC							
000001ED							
000001EE							
000001EF							

- 栈区的空间用尽仍不够使用时，出现运行错误
- 堆区的空间用尽仍不够使用时，提示失败

数组的声明

- 格式 `<类型> <数组名>[<数组长度>];`
 - 数组名的值 a 是声明时开辟的内存空间首地址，不可更改。
 - 声明时在类型前加注 `const`，数组元素不可更改。
 - 数组长度 N 应为无符号整型常数，可以为0。
 - 声明语句的方括号是修饰符；而表达式的方括号是操作符。
- 声明数组后开辟的空间：数组元素类型 $\times$ 数组长度
 - 设指定类型占有 t 字节，则开辟的空间为 tN 字节

声明语句	占用字节
<code>int arr[100];</code>	$4*100=400$
<code>double score[50], final_score[50];</code>	$8*50+8*50=800$
<code>const int base[3] = { 1, 2, 4 };</code>	$4*3=12$

数组的初始化

- 声明时，用复合文字初始化

```
int powers[8] = { 1, 2, 4, 6, 8, 16, 32, 64 };  
int powers[] = { 1, 2, 4, 6, 8, 16, 32, 64 };
```

- 数组长度以指定为准；不指定时，以复合文字长度为准

- 声明后，不可使用复合文字初始化

- 循环赋值（不一定要初始化为0）

```
for (i = 0; i < sizeof(arr) / sizeof(arr[0]); i++)  
    arr[i] = 0;
```

- 内存操作函数（注意内存格式）

```
memset(arr, 0, sizeof(arr));
```

- 数组应**先声明再使用**，数组元素应**先写后读**

复合文字（数组）

- 复合文字是按初始值设定项列表初始化的未命名对象。

- 类似于字符串、代码块，值为所占空间的首地址

```
char str[] = "Hello!";
```

72	101	108	108	111	33	0
----	-----	-----	-----	-----	----	---

```
int arr[] = { 1, 2, 4, 6, 8, 16, 32 };
```

1	2	4	6	8	16	32
---	---	---	---	---	----	----

- 格式：元素应为表达式，花括号为起止，逗号分隔

- 指定下标赋值 {1, 2, [4]=4, 6, [1]=16, 32}

1	16	32	0	4	6
---	----	----	---	---	---

- 显式指定下标的元素，以指定为准，后定义覆盖前定义
- 未指定下标时，为左侧相邻元素下标加1（首元素下标为0）
- 未显式指定值的元素默认为0
- 数组长度为其元素中最大下标加1

数组的元素访问

• 格式

运算符名称	格式	示例
下标运算符	<数组名>[<下标>]	arr[1] = 2;
间接寻址运算符	*(<数组名>+<下标>)	*(arr + 1) = 2;

– 数组下标 i 应为**整型表达式**，否则会有编译错误

- 可以为负数，甚至允许超过其边界（避免此做法）
- 数组越界将产生运行错误
- 下标 i 的下限是 0，上限是 $N-1$ ，超过即为越界

数组越界指的对数组元素的访问超过了边界。

• 下标和次序的关系

– 下标是偏移量，从0开始，第 $i+1$ 个元素对应下标为 i

断言

assert

功能	断言：在代码中捕捉程序员的假设或预期条件。	
格式	<code>void assert(int expression);</code>	
参数	<i>expression</i>	运行中预期为真的表达式。该式会被计算。
返回值	成功	无。
头文件	<code>assert.h</code>	
说明	1. 如果<i>expression</i>为真，或者当前非DEBUG模式，程序没有反应；否则，触发异常终止程序。 2. 如果<i>expression</i>有副作用，会生效。	

利用断言检查数组下标越界错误

```
#include <stdio.h>
#include <assert.h>
#define ARR_LENGTH 10
int main()
{
    int a[ARR_LENGTH];
    for (int i = 0; i < ARR_LENGTH; i++)
        a[i] = i * 2;
    for (int i = 0; i < ARR_LENGTH; i += 2)
    {
        assert(i + 1 >= 0 && i + 1 < ARR_LENGTH);
        assert(i + 2 >= 0 && i + 2 < ARR_LENGTH);
        a[i] = a[i + 1] - a[i + 2];
    }
    for (int i = 0; i < ARR_LENGTH; i++)
        printf("a[%d]=%d\n", i, a[i]);
    return 0;
}
```

有经验的程序员制备大量测试数据，运行程序，如果在这里中止，说明该组数据下，数组下标越界。

多维数组的声明

- 格式 `<类型> <数组名>[<数组长度3>][<数组长度2>][<数组长度1>];`

- 以下以声明 `type m[Nd][Nd-1]...[N1]` 为例

- 数组长度 N_i 应为无符号**整型常数**，可以为0。

- 元素个数为各维度的乘积 $\prod_{i=1}^d N_i$

- 高维数组可以视为低维数组为元素的数组

- 较高的维度在前，较低的维度在后 `int matrix[3][5];`

这是3个int m[5]
堆起来的大数组

- 声明时用复合文字进行初始化

- 复合文字可以作为元素，组成大的复合文字。

```
int powers[2][8] = {{1,2,4,6,8,16,32,64},  
                    {1,2,4,6,8,16,32,64}};
```

多维数组的访问

- 格式 `<数组名>[<下标3>][<下标2>][<下标1>];`

- 以下以访问 $m[i_d][i_{d-1}]\dots[i_1]$ 为例

- 其中：数组下标应为**整型表达式**，否则会有编译错误

- 可以为负数，甚至超过其边界（程序员应避免此做法）

- 多维数组的应先计算偏移量再求值

$$\text{偏移量 } I = \sum_{k=2}^d \left(i_k \cdot \prod_{j=1}^{k-1} N_j \right) + i_1,$$

- 偏移量相同的，指向元素的值也相同

- 应该书写含义明确的下标形式

```
int m[3][5];
```

- d : 数组的维度数
- N_j : 第 j 维的长度（元素个数）
- i_k : 访问时第 k 维的下标值
- t : 每个元素的大小（字节数）
- 最后的 i_1 : 最低维的下标，直接加到总偏移量上

这是 $m[1][4]$ ，也是 $m[0][9]$ ，
 $m[2][-1]$, $m[-1][14]$, $m[3][-6]$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----

数组的局限性

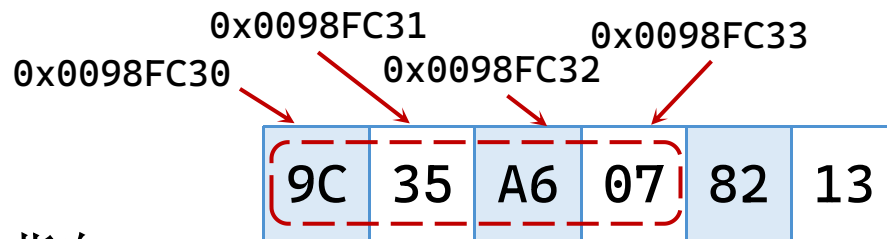
- 数组的局限性

- 数组作为局部变量时，所在栈区空间有限（默认1—8M）
- 数组作为全局变量时，破坏程序的逻辑结构
- 整理代码时，数组名是常量，无法被赋值

- 解决上述问题，需要新的数据类型：指针

- 指针为无符号整数，其物理意义为指向内存地址
- 指针的值为内存地址的偏移量，是内存一段区域的首地址
- 格式为“类型名”的类型、
长度为“类型名”的长度

- $p = 0x0098FC30$ 就意味着 p 指向 9C



题目

2-8

分数 2

[查看题目详情](#)

作者 周强

单位 青岛大学

对于定义 `char str[] = "abc\000def\n"`（注：其中0为数字零），求字符串str的长度len和数组str的大小size，len和size分别是：

- ☐ A. 12 , 13
- ☐ B. 3 , 11
- ☐ C. 3 , 9
- ☐ D. 3 , 4

C

字符串 str 的内容为 "abc\000def\n"，其中 \000 是八进制转义字符，表示 ASCII 码为 0 的字符（即空字符 \0），它会终止字符串。因此字符串实际有效长度从开头到第一个 \0 结束，即 abc，长度为 3。数组大小包括所有字符直到字符串结束符，但 \000 后的 def\n 不会被计入有效字符串，且数组大小需包含所有字节。 \000 占一个字节，def\n 共 4 个字符，加上前面的 abc 和 \0，加上字符串末尾 \0 总共 9 个字符。所以字符串长度 len 为 3，数组大小 size 为 9，答案是 C。

题目

2-17 分数 2 查看题目详情

以下程序的输出结果是（ ）。

```
int main(void)
{
    int m[ ][3] = { 1, 4, 7, 2, 5, 8, 3, 6, 9 };
    int i, j, k=2;

    for (i=0; i<3; i++)
        printf ("%d ", m[k][i]);

    return 0;
}
```

C。

m[][3]按行存成 3×3 数组：{{1,4,7}, {2,5,8}, {3,6,9}}。k=2 选中第 3 行，即 m[2] 为 3,6,9，循环中 i=0~2 输出 m[2][0], m[2][1], m[2][2]，所以结果是 3 6 9。

- ☐ A. 4 5 6
- ☐ B. 2 5 8
- ☐ C. 3 6 9
- ☐ D. 7 8 9

题目

2-24 分数 2 查看题目详情

作者 周强 单位 青岛大学

对于定义 `char str[] = "abc\0def";`（注：其中 `0` 为数字零），求字符串 `str` 的长度 `len`（即调用标准库函数 `strlen`：`len = strlen(str)`）和数组 `str` 的大小 `size`（即 `size = sizeof(str)`），`len` 和 `size` 分别是：

- ☐ A. 7, 8
- ☐ B. 7, 7
- ☐ C. 3, 8
- ☐ D. 3, 7

C。

"abc\0def"里实际字符是：a b c \0 d e f \0，一共 8 个字节，所以 `sizeof(str)` 等于 8；
`strlen(str)` 从开头数到第一个 `\0` 停止，只能数到 abc，所以长度是 3。

10

函数

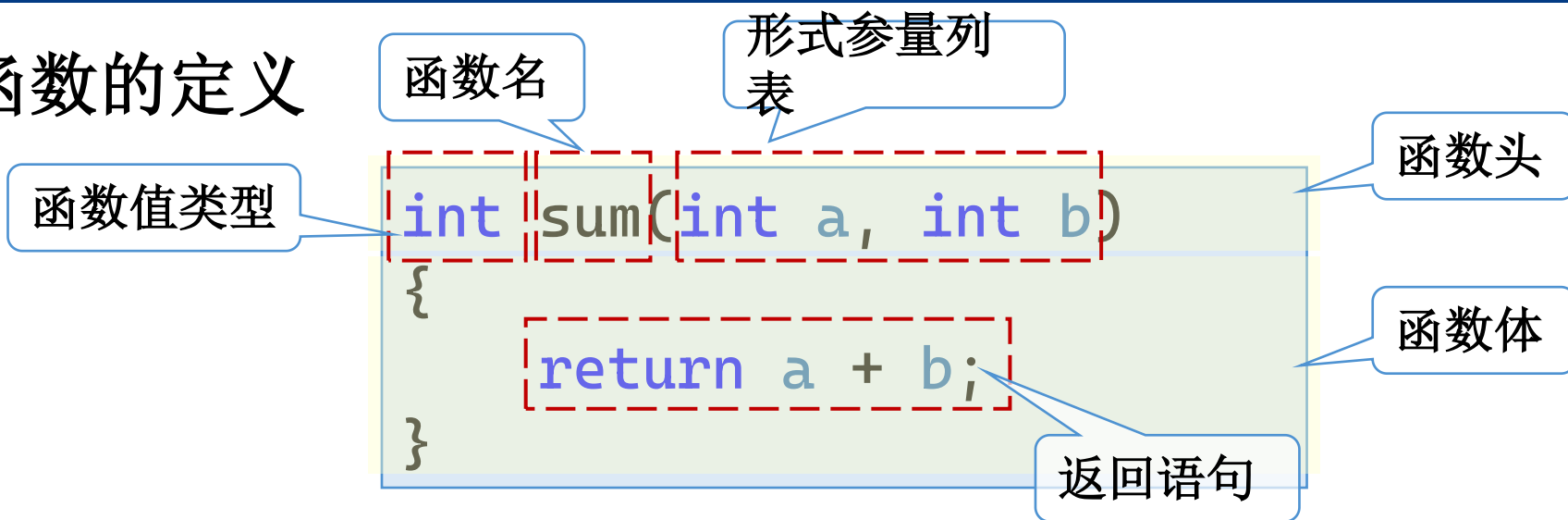
理论课程



福州大学
FUZHOU UNIVERSITY

函数的组成

- 函数的定义



- 函数的声明

```
int sum(int a, int b);
```

- 函数的调用

```
c = sum(a, 2) * 3;
```

参数列表

函数表达式的值

```

/* lethead1.c */
#include <stdio.h>
#define NAME "GIGATHINK, INC."
#define ADDRESS "101 Megabuck Plaza"
#define PLACE "Megapolis, CA 94904"
#define WIDTH 40
void starbar(void); /* prototype the function */

```

函数的声明，函数应：先声明、有定义、再调用

```

int main(void) {
    starbar();
    printf("%s\n", NAME);
    printf("%s\n", ADDRESS);
    printf("%s\n", PLACE);
    starbar(); /* use the function */

    return 0;
}

```

函数的调用

```

void starbar(void) /* define the function */
{
    int count;

    for (count = 1; count <= WIDTH; count++)
        putchar('*');
    putchar('\n');
}

```

函数的定义
这是个没有参量的函数

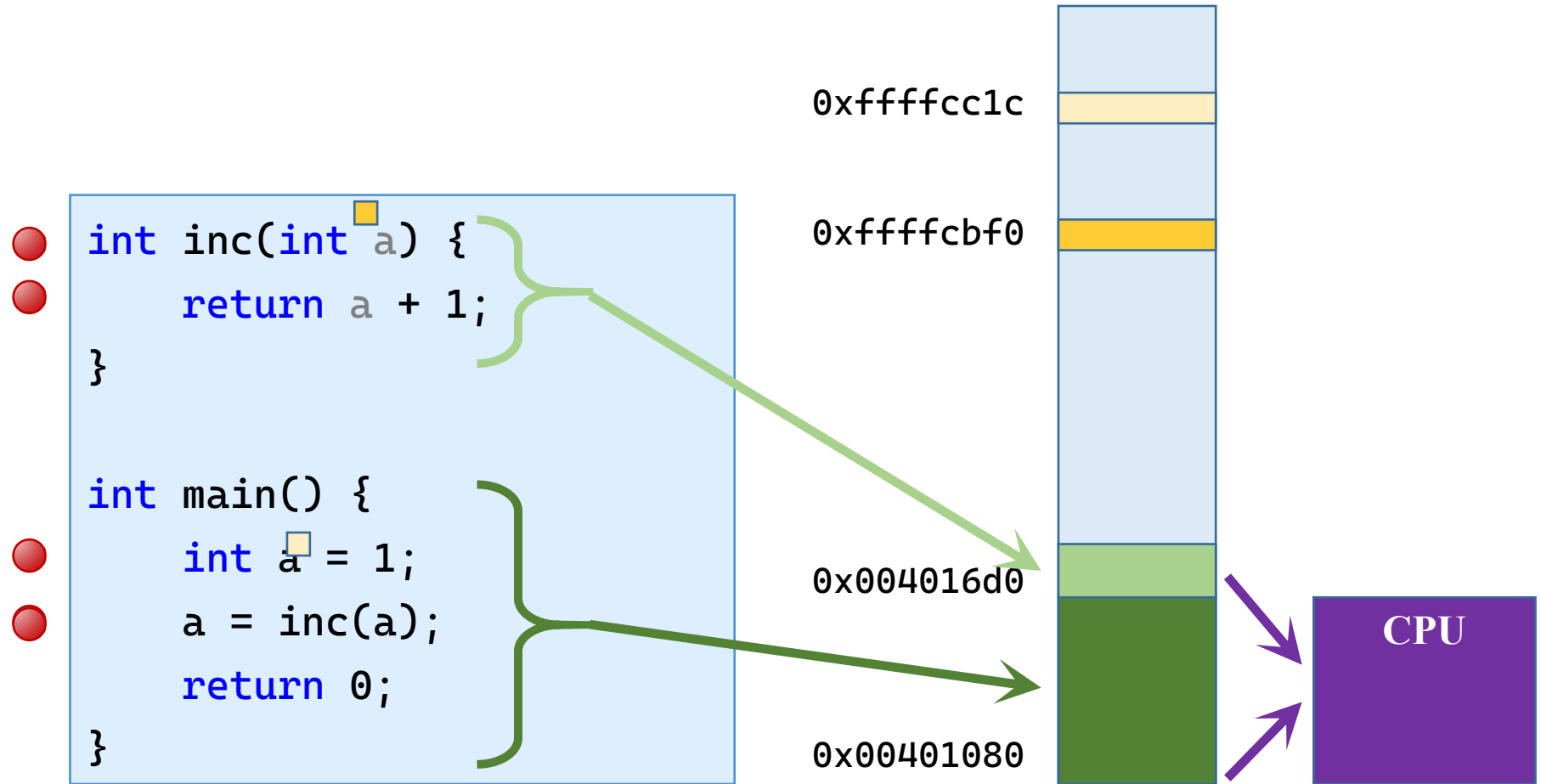
```

*****
GIGATHINK, INC.
101 Megabuck Plaza
Megapolis, CA 94904
*****

```

函数调用 (Function call)

- 函数调用的执行过程和内存内容



函数参量和参数

- 函数参量称为形式参量
 - 函数调用者向被调用者传递的信息的量
- 实际参数
 - 函数在调用时通过实际参数对形式参量进行赋值

```
int main(void)
{
    // ...
    show_n_char('*', 20);
    // ...
    return 0;
}

void show_n_char(char* ch, int num)
{
}
```

The diagram illustrates the relationship between a function call and its definition. On the left, the `main` function is shown with a call to `show_n_char('*', 20);`. On the right, the `show_n_char` function is defined with parameters `char* ch` and `int num`. Blue arrows show the mapping: one arrow points from the opening curly brace of `main` to the opening curly brace of `show_n_char`, and another arrow points from the closing curly brace of `main` back to the closing curly brace of `show_n_char`. Dashed green boxes highlight the parameter lists in both functions.

递归函数（Recursive Functions）

- 递归函数是调用自身的函数

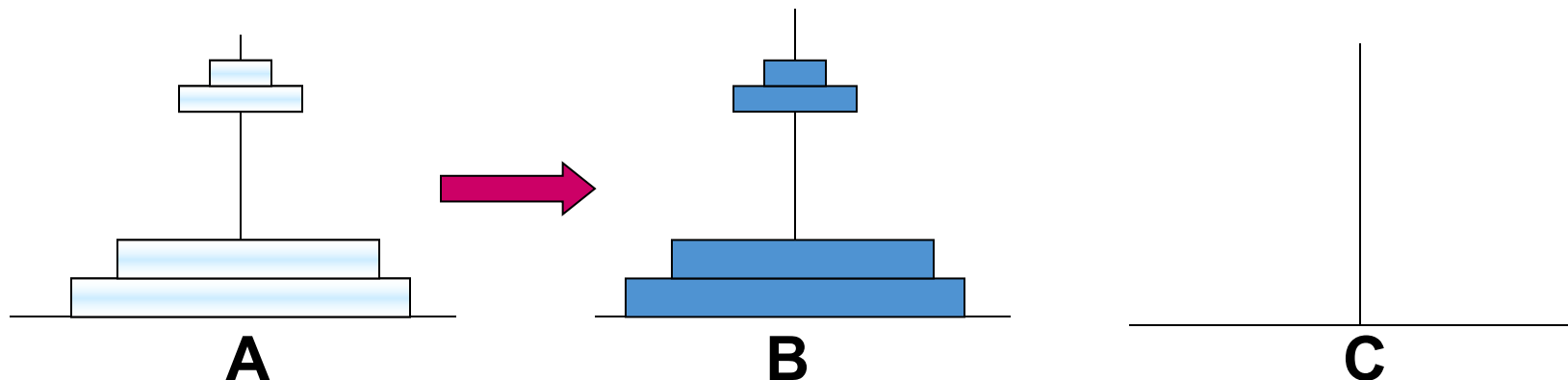
- 每一级的函数调用都有自己的变量，但共用一个代码段
- 每次调用都会有一次返回

- 函数的调用顺序

- 递归调用之前顺序执行
- 调用语句之后逆序执行

参数	各层变量n的值			
	第1层	第2层	第3层	第4层
调用第1层之后	1	*	*	*
调用第2层之后	1	2	*	*
调用第3层之后	1	2	3	*
调用第4层之后	1	2	3	4
第4层返回之后	1	2	3	*
第3层返回之后	1	2	*	*
第2层返回之后	1	*	*	*
第1层返回之后	*	*	*	*

汉诺(Hanoi)塔

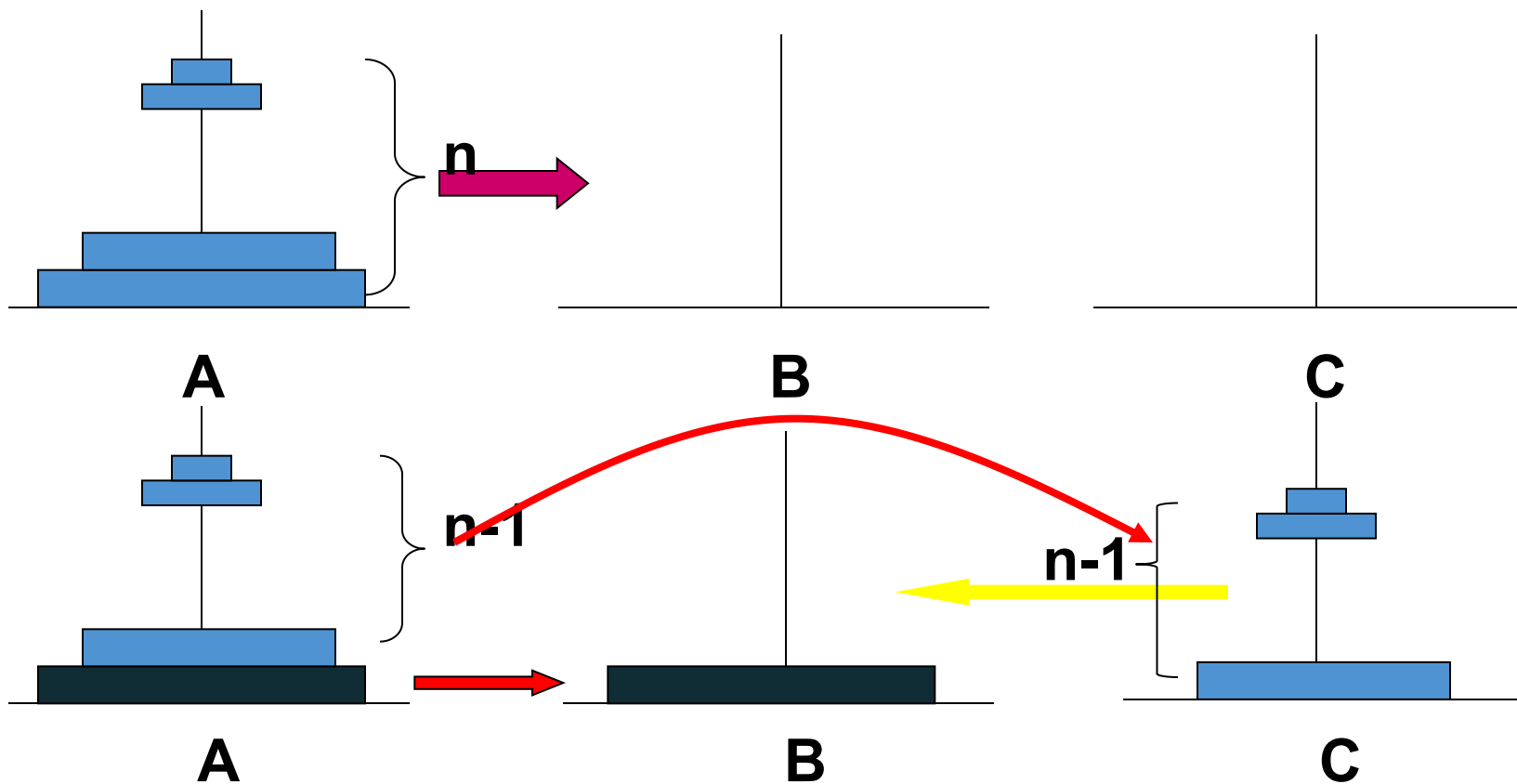


将 **64** 个盘从座**A**搬到座**B**

- (1) 一次只能搬一个盘子
- (2) 盘子只能插在**A**、**B**、**C**三个杆中
- (3) 大盘不能压在小盘上

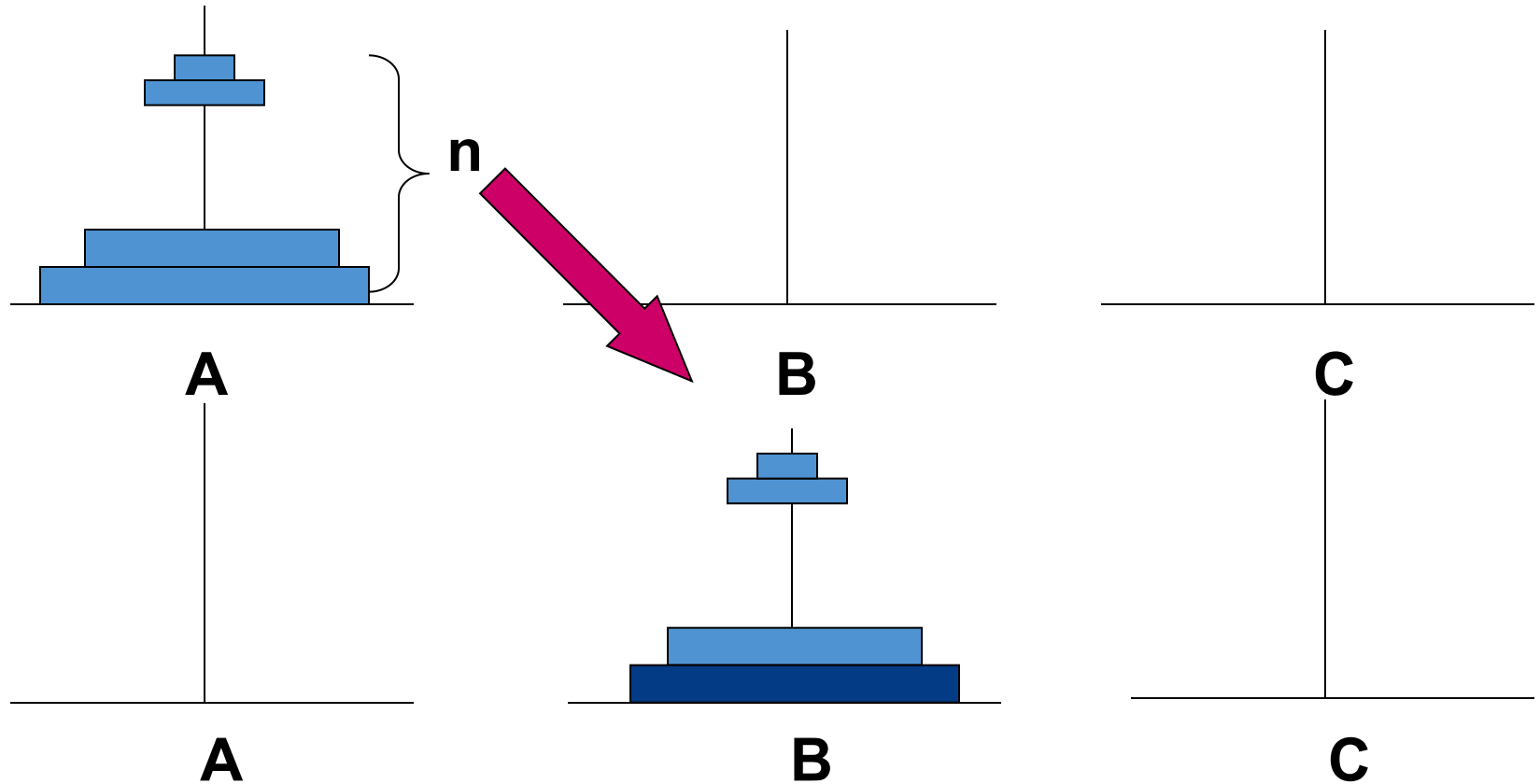
汉诺(Hanoi)塔

分析



汉诺(Hanoi)塔

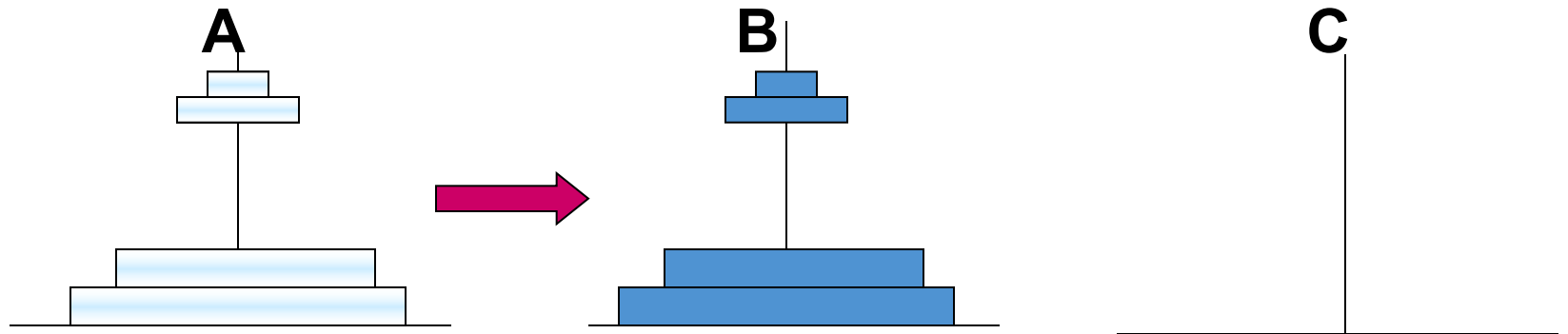
分析



汉诺(Hanoi)塔

- 递归方法的两个要点
 - (1) 递归出口：一个盘子的解决方法；
 - (2) 递归式子：如何把搬动**64**个盘子的的问题简化成搬动**63**个盘子的的问题。
- 把汉诺塔的递归解法归纳成三个步骤：
 - **n-1**个盘子从座**A**搬到座**C**
 - 第**n**号盘子从座**A**搬到座**B**
 - **n-1**个盘子从座**C**搬到座**B**

汉诺(Hanoi)塔



```
hanio(n个盘, A→B, C为过渡) {  
    if (n == 1)  
        直接把盘子A→B  
    else {  
        hanio(n-1个盘, A→C, B为过渡)  
        把第n号盘 A→B  
        hanio(n-1个盘, C→B, A为过渡)  
    }  
}
```

```

#include <stdio.h>
void hanio(int n, char a, char b, char c)
{
    if (n == 1)
        printf("%c-->%c\n", a, b);
    else
    {
        hanio(n-1, a, c, b);
        printf("%c-->%c\n", a, b);
        hanio(n-1, c, b, a);
    }
}
int main(void)
{
    int n;
    scanf("%d", &n);
    hanio(n, 'A', 'B', 'C') ;
    return 0;
}

```



```
#include <stdio.h>
```

```
void hanio(int n, char a, char b, char c)
```

```
{
    if (n == 1)
        printf("%c-->%c\n", a, b);
    else
    {
        hanio(n-1, a, c, b);
        printf("%c-->%c\n", a, b);
        hanio(n-1, c, b, a);
    }
}
```

```
int main(void)
```

```
{
    int n;
    scanf("%d", &n);
    hanio(n, 'A', 'B', 'C') ;
    return 0;
}
```

初始:

A: [3,2,1] B: [] C: []

第1步: A→B (盘1)

A: [3,2] B: [1] C: []

第2步: A→C (盘2)

A: [3] B: [1] C: [2]

第3步: B→C (盘1)

A: [3] B: [] C: [2,1]

第4步: A→B (盘3 最大盘)

A: [] B: [3] C: [2,1]

第5步: C→A (盘1)

A: [1] B: [3] C: [2]

第6步: C→B (盘2)

A: [1] B: [3,2] C: []

第7步: A→B (盘1)

A: [] B: [3,2,1] C: []

hanio(3, A, B, C)

hanio(2, A, C, B)

hanio(1, A, B, C) → 打印 A-->B (最小盘)

打印 A-->C (第二大盘)

hanio(1, B, C, A) → 打印 B-->C (最小盘)

打印 A-->B (最大盘!)

hanio(2, C, B, A)

hanio(1, C, A, B) → 打印 C-->A (最小盘)

打印 C-->B (第二大盘)

hanio(1, A, B, C) → 打印 A-->B (最小盘)

递归的优缺点

- 优点

- 为某些编程问题提供了最简单的解决方法，结构优美

- 缺点

- 递归算法耗费计算机内存资源，限制计算规模，效率较低

- 解决方案

- 自己实现“栈”，选择最少的Push和Pop操作（不保存 CPU 寄存器状态、返回地址）

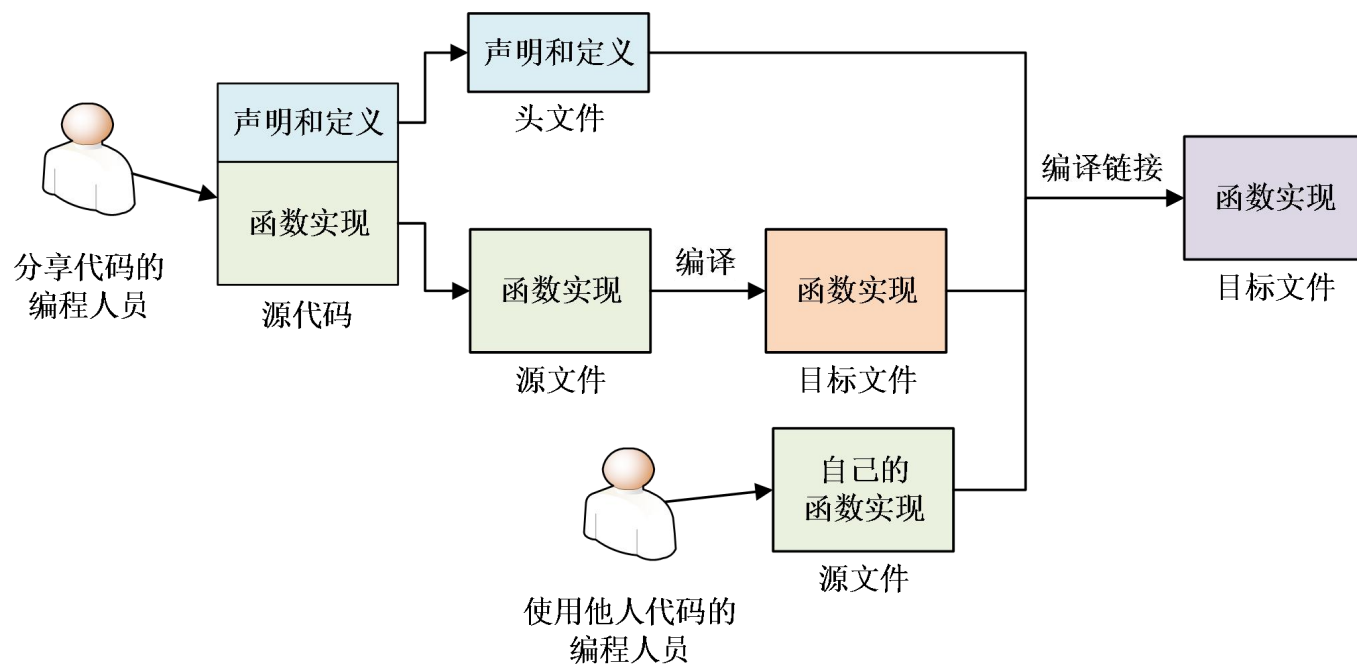
- 设计矩阵保存已知的计算结果，免得下次再算

函数复用

- 书写函数提供他人使用，需要提供声明和实现
- 共享函数的声明
 - 函数原型：通常存于头文件 (*.h) 中
- 共享函数的实现
 - 有的程序员开放源代码共享
 - 将函数源代码归类于源代码文件 (*.c) 中
 - 有的程序员掩盖技术秘密
 - 将源代码编译为目标文件 (*.obj)，打包为库 (*.lib) 文件

分发过程（GCC）

- 在提供软件包的一方将源代码编译为目标文件
 - 将得到的ext.o与ext.h分发出去
- 使用软件包的一方
 - 包含文件头



```
// ext.c -- an external code
#include <stdio.h>
```

```
int print_msg(const char* message)
{
    int ret;
    ret = printf("In ext.c, print_msg(\"%s\")\n",
message);
    printf("In ext.c, print_msg: RETURN %d\n", ret);
    return ret;
}
```

```
void prt_msg(const char* message)
{
    printf("In ext.c, prt_msg(\"%s\")\n", message);
}
```

函数的实现
在此处包含或不包含原型
声明都可以
(不会造成编译错误即可)

```
// ext.h -- a header file of ext.c  
int print_msg(const char* message);  
void prt_msg(const char* message);
```

函数的设计者应提供函数的原型声明

```
// main.c - a code to test ext.c
```

```
#include <stdio.h>
```

```
#include "ext.h"
```

```
int main()
```

```
{
```

```
    int getret;
```

```
    printf("In main.c, main()\n");
```

```
    printf("In main.c, CALL print_msg()\n");
```

```
    getret = print_msg("This is a message");
```

```
    printf("In main.c, print_msg() RETURN %d\n", getret);
```

```
    printf("In main.c, CALL prt_msg()\n");
```

```
    prt_msg("This is a message");
```

```
    printf("In main.c, main()\n");
```

```
    return 0;
```

```
}
```

使用者包含该头文件，
使用函数的原型声明

声明可以直接用

题目

2-5 分数 2 查看题目详情

下列程序的输出结果是 () 。

```
fun(int a, int b, int c)
{   c = a * b;   }
int main(void)
{
    int c;

    fun(2, 3, c);
    printf("%d\n", c);

    return 0;
}
```

- ☐ A. 0
- ☐ B. 1
- ☐ C. 6
- ☐ D. 无法确定

D。

main里的c没有初始化，值本来就是不确定的；fun(2, 3, c)也是按值传递，只在函数内部改了形参c，并没有改动main里的那个c，所以printf打印一个未初始化的局部变量，结果是未定义的，无法确定。

题目

4-1 分数 4 查看题目详情

以下程序的输出结果是

4分。

```
1 void fun(int x, int y) {  
2     x = x + y;  
3     y = x - y;  
4     x = x - y;  
5     printf("%d, %d, ", x, y);  
6 }  
7  
8 int main(void) {  
9     int x = 2, y = 3;  
10    fun(x, y);  
11    printf("%d, %d\n", x, y);  
12    return 0;  
13 }
```

fun(x, y) 里形参是实参的副本，只在函数内部交换了 x 和 y，于是先打印出 3, 2,; 函数结束后回到 main，原来的 x 和 y 仍然是 2 和 3，再打印 2, 3，所以总输出为 3, 2, 2, 3。

11

存储类、链接 和内存管理

理论课程



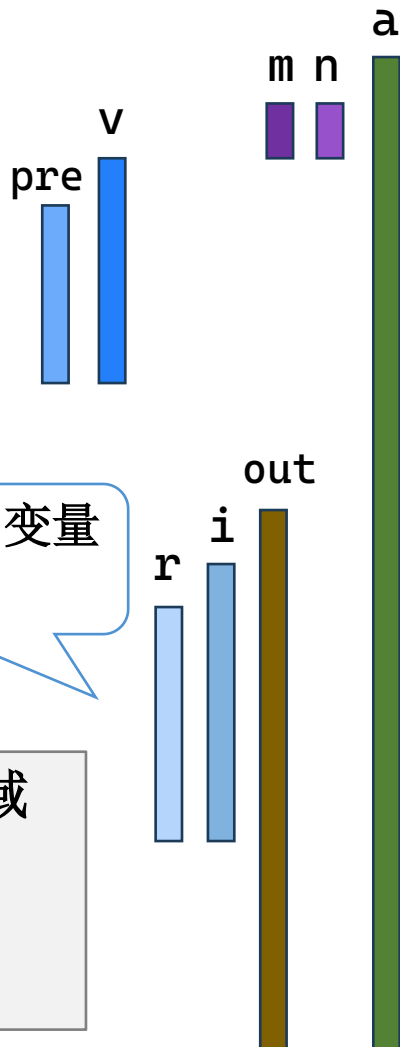
福州大学
FUZHOU UNIVERSITY

作用域

- 代码块作用域(**block scope**)
 - 默认在声明语句之后且在声明语句的代码块内使用
- 文件作用域(**file scope**)
 - 在所有函数之外声明的变量或函数，在该文件内使用
- 函数原型作用域 (**function prototype scope**)
 - 函数声明语句中出现的参量，在声明语句内使用
- 函数作用域 (**function scope**)
 - 函数中的标签对于函数内部可见，外部不可goto进来

作用域

• 示例



```
#include <stdio.h>
int a = 0;
void sum(int m, int n, int r[m][n]);
int amplify(int* v) {
    int pre = *v;
    *v *= a;
    return pre;
}
void sum(int x, int y, int b[x][y]) {
    // omitted...;
}
int main() {
    for (int i = 0; i < 10; i++) {
        int r = amplify(&i);
        a = a + i - r;
        if (i % 5 == 3)
            goto out;
    }
out:
    printf("%d", a);
    return 0;
}
```

- 代码块作用域
- 文件作用域
- 原型作用域
- 函数作用域

链接

- 空链接 (no linkage)
 - 具有代码块作用域或函数原型作用域的变量
 - 由其所在代码块或函数原型所私有
- 外部链接 (external linkage)
 - 外部链接的变量可以在多文件程序的任何地方使用
- 文件作用域变量具有内部或外部链接
- 内部链接 (internal linkage)
 - 只能在当前文件的任何地方使用

链接

• 示例

```
#include <stdio.h>
extern int s = 0;
void print() {
    printf("%d\n", s);
}
```

- 空链接: v, pre, i, r
- 外部链接: s
- 静态链接: a

a

s

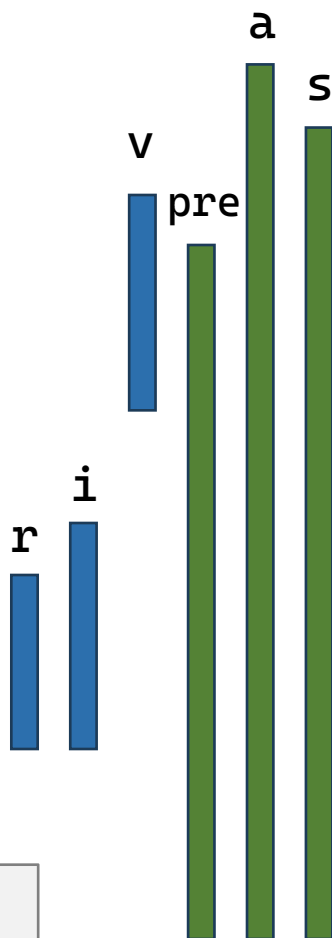
```
#include <stdio.h>
static int a = 0;
int s = 0;
int amplify(int* v) {
    int pre = *v;
    *v *= a;
    return pre;
}
int main() {
    for (int i = 0; i < 10; i++) {
        int r = amplify(&i);
        a = a + i - r;
    }
    printf("%d", a);
    return 0;
}
```

存储时期

- 静态存储时期 (static storage duration)
 - 文件作用域的变量具有静态存储时期
 - 运行声明语句时分配内存；退出程序前不释放。
 - 注意：标注 `static` 不表示静态存储时期，而是链接类型
- 自动（动态）存储时期 (auto storage duration)
 - 代码块作用域的变量一般具有自动存储时期
 - 进入代码块时分配内存；退出则释放。

存储时期

• 示例



■ 自动存储时期
■ 静态存储时期

```
#include <stdio.h>
static int a = 0;
int s = 0;
int amplify(int* v) {
    static int pre = *v;
    *v *= a;
    return pre;
}
int main() {
    for (int i = 0; i < 10; i++) {
        int r = amplify(&i);
        a = a + i - r;
    }
    printf("%d", a);
    return 0;
}
```


存储类

• 5种存储类

作用域	链接	自动生存期	静态生存期
代码块	空链接	1. 自动类 2. 寄存器类	3. 空链接的静态类
文件	内部链接	/	4. 具有内部链接的静态类
	外部链接	/	5. 具有外部链接的静态类

存储类	声明语句示例
1. 自动存储类	<code>auto int a;</code> （限函数内）
2. 寄存器存储类	<code>register int r;</code> （限函数内）
3. 空链接的静态存储类	<code>static int sn;</code> （限函数内）
4. 具有外部链接的静态存储类	<code>int e;</code> （限全局变量）
5. 具有内部链接的静态存储类	<code>static int si;</code> （限全局变量）

自动类

- 声明自动类变量

- 位置：函数内，包括函数参量列表

说明	格式	示例
显式声明	auto <类型> <变量名>	auto int a;
隐式声明	<类型> <变量名>	int a;

- 作用域：代码块；链接：空链接（代码块内）

- 生存期：自动生存期（函数结束时销毁）

- 规则

- 内层定义名称与外层定义相同时，内层自动覆盖外层

- 自动变量不会自动初始化，需自行初始化

寄存器类

- 声明寄存器类变量

- 位置：函数内，包括函数参量列表

说明	格式	示例
显式声明	<code>register <类型> <变量名></code>	<code>register int r;</code>

- 作用域：代码块；链接：空链接（代码块内）

- 生存期：自动生存期（函数结束时销毁）

- 规则

- **请求**变量存储在寄存器或高速内存中，提供高速访问

- 可声明的类型和数量有限，请求不到也没办法

- 不可以取内存地址（**寄存器在CPU里**）

代码块作用域的静态类

- 声明代码块作用域的静态类变量（静态局部变量）

- 位置：函数内，**不**包括函数参量列表

说明	格式	示例
显式声明	<code>static <类型> <变量名></code>	<code>static int r;</code>

- 作用域：文件；链接：空链接（代码块内）

- 生存期：静态生存期（程序结束时销毁）

- 规则

- 初始化：如果声明时未初始化，则初始化为0。

- 先声明后赋值，每次运行重新赋值

```
static int val; val = 0;
```

- 声明时赋值，只在第一次运行赋值

```
static int val = 0;
```

注意：静态是指内存位置的静态，不是值的静态

```

/* loc_stat.c -- using a local static variable */
#include <stdio.h>
void trystat(void);
int main(void)
{
    int count;
    for (count = 1; count <= 3; count++)
    {
        printf("Here comes iteration %d:\n", count);
        trystat();
    }
    return 0;
}

void trystat(void)
{
    int fade = 1;
    static int stay = 1;
    printf("fade = %d and stay = %d\n", fade++, stay++);
}

```

Here comes iteration 1:
 fade = 1 and stay = 1
 Here comes iteration 2:
 fade = 1 and stay = 2
 Here comes iteration 3:
 fade = 1 and stay = 3

外部链接的静态类

- 声明外部链接的静态类变量

- 位置：文件内，所有函数之外

说明	格式	示例
定义声明	<类型> <变量名>	<code>int r;</code>
引用声明	<code>extern</code> <类型> <变量名>	<code>extern int r;</code>

- 作用域：文件；链接：外部链接（外部代码可用）

- 生存期：静态生存期（程序结束时销毁）

- 规则

- 在一个文件的全局变量区域做定义声明

- 使用该全局变量的其他文件里做引用声明

内部链接的静态类

- 声明内部链接的静态类变量

- 位置：文件内，所有函数之外

说明	格式	示例
定义声明	<code>static <类型> <变量名></code>	<code>static int r;</code>
引用声明	<code>extern <类型> <变量名></code>	<code>extern int r;</code>

- 作用域：文件；链接：内部链接（外部代码不可用）

- 生存期：静态生存期（程序结束时销毁）

- 规则

- 在一个文件的全局变量区域做定义声明

- 限本文件该用到该全局变量的函数做引用声明（不必要）

```
// parta.c --- various storage classes
// compile with partb.c
#include <stdio.h>
void report_count();
void accumulate(int k);
int count = 0;           // file scope, external linkage
int main(void) {
    int value;           // automatic variable
    register int i;      // register variable
    printf("Enter a positive integer (0 to quit): ");
    while (scanf("%d", &value) == 1 && value > 0) {
        ++count;        // use file scope variable
        for (i = value; i >= 0; i--)
            accumulate(i);
        printf("Enter a positive integer (0 to quit): ");
    }
    report_count();
    return 0;
}

void report_count() {
    printf("Loop executed %d times\n", count);
}
```

```
Enter a positive integer (0 to quit): 6↵
loop cycle: 1
subtotal: 21; total: 21
Enter a positive integer (0 to quit): 9↵
loop cycle: 2
subtotal: 45; total: 66
Enter a positive integer (0 to quit): a↵
Loop executed 2 times
```



```

// partb.c -- rest of the program
// compile with parta.c
#include <stdio.h>
extern int count;    // reference declaration, external linkage
static int total = 0;    // static definition, internal linkage
void accumulate(int k); // prototype
void accumulate(int k) { // k has block scope, no linkage
    static int subtotal = 0; // static, no linkage
    if (k <= 0) {
        printf("loop cycle: %d\n", count);
        printf("subtotal: %d; total: %d\n", subtotal, total);
        subtotal = 0;
    }
    else {
        subtotal += k;
        total += k;
    }
}
}

```

存储类和函数

分类	外部函数	内部函数
声明格式	<类型> <函数名>(<参量列表>);	static <类型> <函数名>(<参量列表>);
声明示例	<code>double gamma();</code>	<code>static double gamma();</code>
使用范围	其它文件可用	限同一文件范围内使用
默认值	是	不是

动态内存分配

calloc

功能	动态内存分配，并初始化为0。	
格式	<code>void* calloc(size_t num, size_t size);</code>	
参数	<i>num</i>	对象数目。
	<i>size</i>	每个对象的长度，单位：字节。
返回值	成功	指向新分配内存开头的指针
	失败	NULL
示例	<code>p = (int *)calloc(6, sizeof(int));</code>	
头文件	<code>stdlib.h</code>	
说明	1. 如果<i>num</i>或<i>size</i>为0，则返回值未定义。 2. 因为对齐需求，分配的字节数不必等于 <i>num*size</i>。 3. 分配存储中的所有字节初始化为0。 4. 指针不使用时，应使用<code>free()</code>释放，避免内存泄漏。	

内存填充

memset

功能	内存填充	
格式	<code>void *memset(void *dest, int ch, size_t count);</code>	
参数	<i>dest</i>	指向要填充的对象的指针
	<i>ch</i>	填充字节值
	<i>count</i>	要填充的字节数
返回值	成功	指针 <i>dest</i> 的副本。
示例	<code>memset(p, 0, 6);</code>	
头文件	<code>string.h</code>	
说明	无。	

动态内存释放

free

功能	归还之前已经分配的内存。	
格式	<code>void free(void* ptr);</code>	
参数	<i>ptr</i>	指针。之前由malloc()、calloc()、realloc()或aligned_alloc()分配的空间。
返回值	成功	无
示例	<code>free(p);</code>	
头文件	stdlib.h	
说明	1. 如果<i>ptr</i>为NULL，函数不进行操作；<i>ptr</i>不等于之前分配的任何空间，或该空间已归还，行为未定义。 2. 归还空间后，该空间被分配给其他变量前，通过该指针访问内存，行为未定义。	

```
/* dyn_arr.c -- dynamically allocated array */
#include <stdio.h>
#include <stdlib.h> /* for malloc(), free() */
int main(void) {
    double* ptd;
    int max;
    int number;
    int i = 0;
    puts("What is the maximum number of type double entries?");
    if (scanf("%d", &max) != 1) {
        puts("Number not correctly entered -- bye.");
        exit(EXIT_FAILURE);
    }
    ptd = (double*)malloc(max * sizeof(double));
    if (ptd == NULL) {
        puts("Memory allocation failed. Goodbye.");
        exit(EXIT_FAILURE);
    }
}
```

```

/* ptd now points to an array of max elements */
puts("Enter the values (q to quit):");
while (i < max && scanf("%lf", &ptd[i]) == 1)
    ++i;
printf("Here are your %d entries:\n", number = i);
for (i = 0; i < number; i++) {
    printf("%7.2f ", ptd[i]);
    if (i % 7 == 6)
        putchar('\n');
}
if (i % 7 != 0)
    putchar('\n');
puts("Done.");
free(ptd);
return 0;
}

```

What is the maximum number of type double entries?

52↓

Enter the values (q to quit):

35↓

99↓

2↓

q↓

Here are your 3 entries:

35.00 99.00 2.00

Done.

分配内存：动态内存 vs 变长数组

分类	内存管理方式	空间限制
动态内存	程序员手动开辟和释放	堆空间
变长数组	系统按生存周期管理	堆空间
定长数组（全局）	系统按生存周期管理	堆空间
定长数组（局部）	系统按生存周期管理	栈空间

题目

2-17 分数 2 查看题目详情

C语言中静态变量和外部变量的初始化是在__阶段完成的。

- ☐ A. 程序执行过程中
- ☐ B. 编译
- ☐ C. 运行
- ☐ D. 程序外部

选 B“编译”，因为静态变量和外部变量属于静态存储区，它们的存储空间和初始值在编译阶段就已经确定下来，并写进可执行文件的数据段中，程序一装入内存时就已经是初始化后的值，所以说是在编译阶段完成初始化。

题目

4-2 分数 4 查看题目详情

写出以下程序的运行结果。

```
#include<stdio.h>
int f(int n)
{
    static int k = 1;

    k++;
    return 2*n + k;
}
int main(void)
{
    printf("%d\n", f(4));
    printf("%d\n", f(f(4)));

    return 0;
}
```

k 是函数内的静态局部变量，只在程序开始时初始化一次，以后每次调用 f 都沿用并修改同一个 k 。

第一次调用 $f(4)$ ：初值 $k=1$ ，执行 $k++$ 得 $k=2$ ，返回 $2*4+2=10$ 。

第二次执行 $f(f(4))$ ：先算内层 $f(4)$ ，此时 $k=2$ ， $k++$ 后 $k=3$ ，返回 $2*4+3=11$ ；再算外层 $f(11)$ ，此时 $k=3$ ， $k++$ 后 $k=4$ ，返回 $2*11+4=26$ 。

第1行输出：

2分

第2行输出：

2分



福州大学

FUZHOU UNIVERSITY

指针

理论课程



福州大学
FUZHOU UNIVERSITY

指针

- 物理意义：指向内存地址（address）
- 内存空间占用：4或8字节，由应用程序的架构决定
- 取值范围：无符号整数
 - 理论上，最小值：0；最大值： $2^{32}-1$ 或 $2^{64}-1$ （程序架构）
 - 实际上，为当前程序可以访问的地址范围
- 指针的属性
 - 指针指向的数据类型
 - 指针的值
 - 指针所在的地址

```
int *p = NULL;
```

这里声明一个指针。
指向数据类型为整型。
指针的值为0。（初始化）
指针的地址为0xffffcc18。（每次运行不同）

指针修饰符

* 修饰符

功能	标记声明的变量为指针		
格式	type *var;		
参数	type	类型名，指针指向的数据类型	
	var	变量名或变量列表，待声明的指针变量	
示例	int *p;		声明指向整型的指针变量p。
	int *p = NULL;		声明整型指针变量p赋值为NULL。
	int *p, *q, r;		声明整型指针变量p和q，和整型变量r。
	int* p, q, r;		*是修饰p而非int，不推荐和int连写。
说明	1. 列表形式声明语句，每个指针前应单独书写修饰符。 2. 声明不应该把int*连着写，以免读者误认p,q为int*类型。		

取址操作符

& 操作符

功能	求地址	
格式	<i>&function</i> 或 <i>&lvalue</i> 或 <i>&refer</i>	
参数	<i>function</i>	函数名
	<i>lvalue</i>	左值，位域和 <code>register</code> 存储类除外
	<i>refer</i>	间接引用，为指针的解引用或数组下标运算
示例	<code>&printf</code>	求printf函数或变量x的地址
	<code>&*p</code>	求p指向内存的地址，即p的值但不是左值
	<code>&a[3]</code>	求数组a第4个元素的地址，即a+3，允许越界
说明	1. 两次运行的结果不一定相同；不同程序架构和编译器下，一些结果的规律可能不同。	

```
#include <stdio.h>
```

```
int main() {
```

```
    int x = 10;
```

```
    int *p = &x;
```

```
    printf("%p\n", p);
```

```
    printf("%p\n", &p);
```

```
    printf("%p\n", &*p);
```

```
    return 0;
```

```
}
```

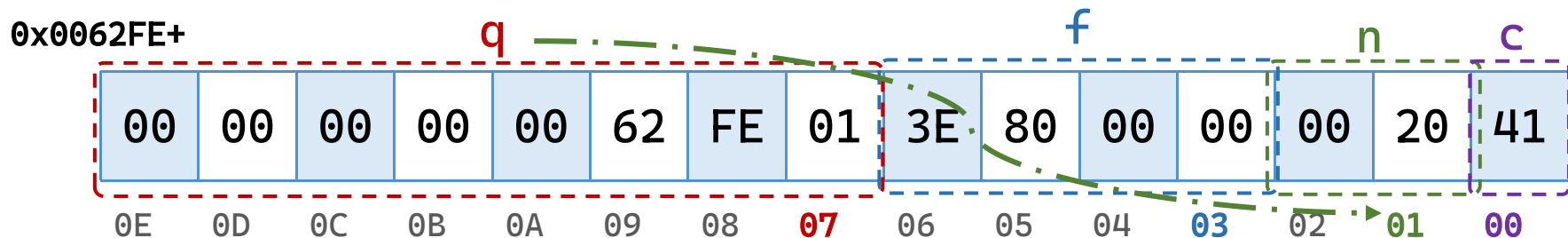
```
00000000000062FE4C
```

```
00000000000062FE40
```

```
00000000000062FE4C
```

声明指针变量时的内存分配

- 指针也是数据类型，指针变量也有所在地址和值



变量名	内存地址	值	长度
c	0x0062FE00	'A'	1B
n	0x0062FE01	32	2B
f	0x0062FE03	0.25	4B
q	0x0062FE07	0x0062FE01	8B

```
char c = 'A';  
short n = 32;  
float f = 0.25;  
short *q = &n;
```


与常量相关指针的声明

- 声明时标记`const`，离其最近的修饰符`*`为只读对象。

示例	<code>p++</code>	<code>(*p)++</code>	<code>(**p)++</code>
<code>int * const p = &q;</code>	✗	✓	/
<code>const int * p = &q;</code>	✓	✗	/
<code>int pq = &q; int * const * p = &pq;</code>	✓	✗	✓
<code>int * const * const p = &pq;</code>	✗	✗	✓
<code>const int * * const p = &pq;</code>	✗	✓	✗
<code>const int * const * const p = &pq;</code>	✗	✗	✗

- 设置`const`后对只读对象赋值，将提示编译错误

[Error] increment of read-only variable 'p'

[Error] increment of read-only location '*(const int * const*)p'

[Error] increment of read-only location '*(const int *)*(const int * const*)p'

解引用操作符

左值 (lvalue)

指的是表示内存中某个可识别位置的表达式，可以出现在赋值运算符 = 的左边。

* 操作符

功能	解引用，将指针值转换为一个左值。	
格式	<i>*pointer</i>	
参数	<i>pointer</i>	指针类型的表达式
返回值	成功	当 <i>pointer</i> 是函数指针时，返回该函数的代指符。 当 <i>pointer</i> 是对象指针时，返回对象的左值表达式。
	失败	解引用空指针、指向在生存期外的对象的指针（悬垂指针）、错误对齐的指针或拥有不确定值的指针是未定义行为。
示例	<i>*p</i>	求 <i>p</i> 指向的对象
说明	1. 无。	

间接寻址和取址操作符

• 例题

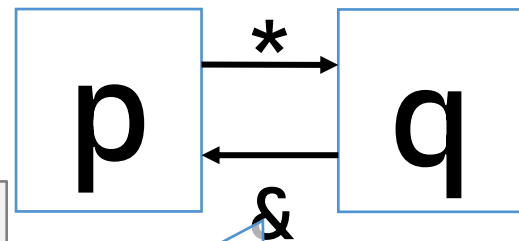
– 执行 `int q = 1, *p = &q;` 后, `*p` 的值为多少?

```
#include <stdio.h>
```

```
int main() {  
    int q = 1, * p = &q;  
    printf("&p=%p\n", &p);  
    printf(" p=%p\n", p);  
    printf("*p=%d\n", *p);  
    printf("&q=%p\n", &q);  
    printf(" q=%d\n", q);  
    return 0;  
}
```

这里*是修饰符, 与表达式中的*不同, 因此运行后p的值为&q的值。
不能认为*p的值为&q的值。

```
&p=0xffffcbcb8  
p=0xffffcbcb4  
*p=1  
&q=0xffffcbcb4  
q=1
```



*p即q, 即q所在空间;
p即&q, 即q所在地址;
&p即p所在地址。

指针指向的数据类型

- 指向类型不相同不能直接赋值
 - 元素类型不同的数组，含高维数组，参量列表不同的函数
- 无类型指针可以赋值给其他指针，再计算

指向数据类型	示例
无类型	<code>void *p;</code>
基本数据类型	<code>char q = 'A'; char *p = &q;</code>
数组	<code>int a[5] = {0}; int *p = a;</code>
指针	<code>int a = 0, *p = &a; int **pp = &p;</code>
结构体	<code>struct student *pstu;</code>
函数	<code>double fabs(double _X); double (*q)(double) = fabs;</code>

无类型指针

- 无类型指针 (`void *`)

- 目标类型未知的情况

- 常用于函数参量，表示任何目标类型的指针都兼容
 - 调用函数时需清楚原始传入的类型，并及时将参量转为对应类型（比如 `float*`、`int*`）。

```
void qsort(void* base, size_t nmem, size_t size,  
           int(*compar)(const void*, const void*));  
int compare(const void* v1, const void* v2) {  
    return *(float*)v1 - *(float*)v2;  
}
```

- 可直接赋值给任何类型指针或被赋值不需强制类型转换
 - 不可以间接寻址或算术运算（目标空间大小未知）

指针和数组的区别联系

- 数组是常量，指针是变量
 - 数组名称的值是该数组元素的首地址，不能被赋值
 - 指针可以用数组赋值
- 指针组成表达式的方法与数组相同
 - 指针间接寻址 $*(a+i)$ 与数组取元素 $a[i]$ 相同
 - 指针间接寻址 $((*a+i)+j)$ 与数组取元素 $a[i][j]$ 相同

```

/* order.c -- precedence in pointer operations */
#include <stdio.h>
int data[2] = {100, 200};
int moredata[2] = {300, 400};
int main(void)
{
    int * p1, * p2, * p3;
    p1 = p2 = data;
    p3 = moredata;
    printf("    *p1 = %d,    *p2 = %d,    *p3 = %d\n",
           *p1,    *p2,    *p3);
    printf("*p1++ = %d, *++p2 = %d, (*p3)++ = %d\n",
           *p1++,    *++p2,    (*p3)++);
    printf("    *p1 = %d,    *p2 = %d,    *p3 = %d\n",
           *p1,    *p2,    *p3);
    return 0;
}

```

*p1 = 100,	*p2 = 100,	*p3 = 300
*p1++ = 100,	*++p2 = 200,	(*p3)++ = 300
*p1 = 200,	*p2 = 200,	*p3 = 301

指向数组的指针作为函数参量

- 指针作为函数的参量

```
int sum(int ar[], int n)
int sum(int * ar) // 也要加 int n
```

 - 两种形式是等价的，ar实际上都是指针
 - 当ar表示数组时，应使用数组形式；当ar表示指针时，应使用指针形式，以免读者误解。
- 函数参量含有数组时，应另有参量指示数组长度
 - 只有在作用域内可通过变量名求出长度
 - 不论何种形式的参量，在函数中都是指针，容易导致越界

在函数外、数组自己定义的地方：
sizeof(a) / sizeof(a[0])

在函数里面都只是 int *，这时候：
sizeof(a) 得到的是指针本身的大小（4/8 字节）

函数用多维数组作为参量

- 函数参量中使用多维数组

- 指出除最高维以外的维度，另设参量为最高维限度

函数形式参量声明	说明
<code>int sum2(int ar[][[]], int rows);</code>	错误，低维长度未指定
<code>int sum2(int ar[][4], int rows);</code>	正确
<code>int sum2(int ar[3][4], int rows);</code>	正确，但最高维长度（3）无效

- 函数中使用数组应正确区分指针和数组

函数形式参量声明	参量ar的含义
<code>int sum4d(int ar[][12][20][30], int rows);</code>	四维数组
<code>int sum4d(int (*ar)[12][20][30], int rows);</code>	指向三维数组的指针

```

/* zippo1.c -- zippo info */
#include <stdio.h>
int main(void) {
    int zippo[4][2] = { { 2, 4 }, { 6, 8 }, { 1, 3 }, { 5, 7 } };

    printf("    zippo = %p,    zippo + 1 = %p\n", zippo, zippo + 1);
    printf("zippo[0] = %p, zippo[0] + 1 = %p\n", zippo[0], zippo[0] + 1);
    printf("    *zippo = %p,    *zippo + 1 = %p\n", *zippo, *zippo +
1);
    printf("zippo[0][0] = %d\n", zippo[0][0]);
    printf("    *zippo[0] = %d\n", *zippo[0]);
    printf("        **zippo = %d\n", **zippo);
    printf("            zippo[2][1] = %d\n", zippo[2][1]);
    printf("*(*(zippo+2) + 1) = %d\n", *(*(zippo+2) + 1));

    return 0;
}

```

```

zippo = 009DF850,    zippo + 1 = 009DF858
zippo[0] = 009DF850, zippo[0] + 1 = 009DF854
*zippo = 009DF850,    *zippo + 1 = 009DF854
zippo[0][0] = 2
*zippo[0] = 2
**zippo = 2
zippo[2][1] = 3
*(*(zippo+2) + 1) = 3

```

指向数组的指针

- 定义指向多维数组的指针 `zippo`的类型是“指向 `int[2]` 的指针”

```
int zippo[4][2];      /* an array of arrays of ints */
```

- 可以被`zippo`赋值的`p_zippo`定义

```
int (*p_zippo)[2];      /* a pointer to the array zippo */
```

- 不可以被`zippo`赋值的`p_zippo`定义

```
int *p_zippo;      /* an improper pointer to the array zippo */  
int **p_zippo;      /* an improper pointer to the array zippo */  
int (*p_zippo)[4];      /* a wrong pointer to the array zippo */  
int *p_zippo[2];      /* a wrong pointer to the array zippo */
```

指向单个 `int` 的指针

指向 `int[4]` 的指针

指向“`int *`”的指针（指针的指针）

含 2 个 `int *` 的数组

利用指针改变函数中变量的值

- 普通参量传递时，无法修改值
 - 变量名只能在声明语句所在的代码块中使用
 - 参量是按值传递，参量值改变，不影响参数值的改变
- 永久改变参量的值，应利用指针
 - 在生命周期内，作用域外修改变量值
 - 参量传入内存地址，函数内操作内存地址，永久修改

利用函数交换两个变量的值

- 左侧代码无法交换，右侧代码可以交换

```
#include <stdio.h>
int swap(int a, int b)
{
    int t;
    t = a; a = b; b = t;
    printf("%d %d\n", a, b);
}
int main()
{
    int a = 3, b = 4;
    printf("%d %d\n", a, b);
    swap(a, b);
    printf("%d %d\n", a, b);
    return 0;
}
```

```
#include <stdio.h>
int swap(int *a, int *b)
{
    int t;
    t = *a; *a = *b; *b = t;
    printf("%d %d\n", *a, *b);
}
int main()
{
    int a = 3, b = 4;
    printf("%d %d\n", a, b);
    swap(&a, &b);
    printf("%d %d\n", a, b);
    return 0;
}
```

将一组数据存储到文件

- 不同类型指针的相互转换

- 常见于和字节数组互转 (`unsigned char *`)

- 常见于开辟空间时 `p = (int *)malloc(sizeof(int) * 60);`

在实际写程序时，经常需要把“某种类型的数据”当成一串原始字节来处理（比如写入文件、发到网络、做加密/压缩等），这时就会在某种类型的指针）和 `unsigned char *`（字节数组指针）之间来回转换，也就是“互转”。

```
struct Point {  
    int x;  
    int y;  
};  
struct Point p = {10, 20};  
// 想把这个结构体按字节写到文件里  
unsigned char *buf = (unsigned char *)&p; //这里就是和 unsigned char* 的互转  
fwrite(buf, 1, sizeof(p), fp);
```

变长数组（C99标准）

- C99允许声明数组时，使用变量指示长度
- 变长数组是声明时用变量表示长度，并非长度可变

```
int quarters = 4;  
int regions = 5;  
double sales[regions][quarters]; // a VLA
```

- 在函数参量中使用变长数组

```
int sum2d(int rows, int cols, int ar[rows][cols]); // ar a VLA
```

```
int sum2d(int ar[rows][cols], int rows, int cols); // invalid order
```

```
int sum2d(int, int, int ar[*][*]); // ar a VLA, names omitted
```

复合文字

- 普通数组的声明方法

```
int diva[2] = {10, 20};
```

- 复合文字

- 没有名称，只能通过赋值给指针（数组不可以）使用

```
(int [2]){10, 20} // a compound literal           “长度未显式给出”的数组类型
(int []){50, 20, 90} // a compound literal with 3 elements
int sum(const int ar[], int n);
...
int total3;
total3 = sum((int []){4,4,4,5,5,5}, 6); // 用符合文字给参数传参
int (*pt2)[4]; // declare a pointer to an array of 4-int arrays
pt2 = (int [2][4]) { {1,2,3,-9}, {4,5,6,-8} }; //多维数组符合文字
```


冒泡排序法 (Bubble Sort)

- 两层循环

- 内层循环逐个比较，将最大值（泡泡）浮至顶部
- 外层循环控制内层循环的范围，已经浮出的元素不需再比较

size	i	a[0]	a[1]	a[2]	a[3]
4	1	4	3	2	1
4	2	3	4	2	1
4	3	3	2	4	1
3	1	3	2	1	4
3	2	2	3	1	4
2	1	2	1	3	4
-	-	1	2	3	4

```
for (size = len; size > 1; size--)  
    for (i = 1; i < size; i++)  
        swap(arr[i - 1], arr[i]);
```

size	i	a[0]	a[1]	a[2]	a[3]
3	1	3	2	1	4

Diagram illustrating the range of the inner loop. A horizontal arrow labeled "size" spans from the column index *i* (which is 1) to the end of the array (index 3). The element at index 4 (value 4) is shown in green, indicating it is sorted and excluded from the current pass.

```

void bubble_sort_1(double arr[], const unsigned int len)
{
    unsigned int size, i;
    for (size = len; size > 1; size--) // 排序的范围从全数组
慢慢收缩至停止
    {
        for (i = 1; i < size; i++) // 单趟排序
        {
            if (arr[i - 1] > arr[i]) // 需要发生交换的条件
            {
                double temp; // temp的唯一正确用法
                temp = arr[i - 1];
                arr[i - 1] = arr[i];
                arr[i] = temp;
            }
        }
    }
}

```

```

void bubble_sort_2(double arr[], const unsigned int len)
{
    unsigned int sorted, i; // sorted是排好序的下标, i是索引
    for (sorted = 0; sorted < len; sorted++) // 排序的范围
        从全数组慢慢收缩至停止, 虽然此处是递增的, 但是在下面使用时有减号
        {
            for (i = 1; i < len - sorted; i++) // 单趟排序
            {
                if (arr[i - 1] > arr[i]) // 需要发生交换的条件
                {
                    double temp; // temp的唯一正确用法
                    temp = arr[i - 1];
                    arr[i - 1] = arr[i];
                    arr[i] = temp;
                }
            }
        }
}

```

冒泡排序法（加强版）

- 增加一个标记量
 - 如果某次内层排序没有实际发生排序，则认定为已经排好序，循环终止。

size	i	a[0]	a[1]	a[2]	a[3]
4	1	4	1	2	3
4	2	1	4	2	3
4	3	1	2	4	3
3	1	1	2	3	4
3	2	1	2	3	4
2	1	1	2	3	4
-	-	1	2	3	4

```

void bubble_sort_3(double arr[], const unsigned int len)
{
    unsigned int size, i; // size是排序的范围, i是索引
    for (size = len; size > 1; size--) // 排序范围从全数组收缩至停
    止
    {
        int is_swapped = 0; // 标记是否交换
        for (i = 1; i < size; i++) // 单趟排序
        {
            if (arr[i - 1] > arr[i]) // 需要发生交换的条件
            {
                double temp; // temp的唯一正确用法
                temp = arr[i - 1];
                arr[i - 1] = arr[i];
                arr[i] = temp;
                is_swapped = 1; // 标记已经经过交换
            }
        }
        if (!is_swapped) // 如果未经交换, 说明已排好序, 可退出不必再排
            break;
    }
}

```

选择排序法 (Select Sort)

- 两层循环

- 内层循环逐个比较，将最大值与所应在位置置换
- 外层循环控制内层循环的范围，已经安排的位置不再比较

pos	min_i	a[0]	a[1]	a[2]	a[3]
0	2	4	3	5	1
1	2	5	3	4	1
2	2	5	4	3	1
3	-	5	4	3	1

pos	j	a[0]	a[1]	a[2]	a[3]
0	1	4	3	5	1

pos+1 len

```
for (pos = 0; pos < len - 1; pos++)  
    for (j = pos + 1; j < len; j++)
```

```
void select_sort(double arr[], const unsigned int len) {  
    for (unsigned int i = 0; i < len - 1; i++) {  
        int min_i = i;  
        for (unsigned int j = i + 1; j < len; j++)  
            if (arr[j] < arr[min_i])  
                min_i = j;  
        if (min_i != i) {  
            double temp = arr[i];  
            arr[i] = arr[min_i];  
            arr[min_i] = temp;  
        }  
    }  
}
```

题目

2-9 分数 2 查看题目详情

根据声明 `int (*p)[10]`, `p` 是一个 ()。

- ☐ A. 数组
- ☐ B. 指针
- ☐ C. 函数
- ☐ D. 数组的元素

B

`int (*p)[10]`

中有括号，先结合成 `*p`，说明 `p` 是一个指针；`[10]` 表示这个指针所指向的是长度为 10 的 `int` 型数组，因此 `p` 是“指向含 10 个 `int` 元素数组的指针”，故选 B。

2-11 分数 2 查看题目详情

在基类型相同的两个指针变量之间，不能进行的运算是 ()。

- ☐ A. `<`
- ☐ B. `=`
- ☐ C. `+`
- ☐ D. `-`

选 C。同类型指针之间可以比较大小 (`<`)、判断是否相等 (`=`)，还可以做减法求两个地址之间相隔的元素个数 (`-`)，但两个地址相加在内存中没有实际意义，C 语言语法上也不允许指针相加运算。

题目

2-12 分数 2 查看题目详情

有如下说明：

```
int a[10]={1, 2, 3, 4, 5, 6, 7, 8, 9, 10}, *P=a;
```

则数值为 9 的表达式是 ()。

- ☐ A. *P+9
- ☐ B. *(P+8)
- ☐ C. *P+=9
- ☐ D. P+8

B。

P 被赋值为 a，即指向数组首元素 a[0]=1；数组中元素 9 在 a[8] 位置，所以用指针形式就是先把指针向后移动 8 个元素得到 P+8，再取值 *(P+8)，因此选 B。

2-13 分数 2 查看题目详情

以下哪个定义中的 p 不是指针，请选择恰当的选项：

- ☐ A. `char **p;`
- ☐ B. `char (*p)[10];`
- ☐ C. `char *p[6];`
- ☐ D. 给出的三项中，p 都是指针

选 C。char **p 中 p 是“指向 char* 的指针”；char (*p)[10] 中 p 是“指向含 10 个 char 的数组的指针”；而 char *p[6] 中 p 是“由 6 个 char* 组成的数组”，p 本身是数组而不是指针。

题目

2-15 分数 2 查看题目详情

下列程序的输出结果是 () 。

```
int main(void)
{
    int a[10] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }, *p = a+3;

    printf("%d", *++p);

    return 0;
}
```

B。

a是 0~9 的数组，p=a+3 让 p 指向 a[3]，也就是元素 3。表达式 *++p 先让指针 p 前移一位，指向 a[4]，再取其值，因此输出的是 a[4] 的值 4。

- ☐ A. 3
- ☐ B. 4
- ☐ C. a[4]的地址
- ☐ D. 非法

下列程序的输出结果是 () 。

```
#include <stdio.h>
#include <string.h>
char *fun(char *t)
{
    char *p = t;
    return (p+strlen(t)/2);
}
int main(void)
{
    char *str = "abcdefgh";

    str = fun(str);
    puts(str);

    return 0;
}
```

C。

字符串 "abcdefgh" 长度为 8， $\text{strlen}(t)/2$ 等于 4， $p+4$ 指向第 5 个字符 'e'，即子串 "efgh" 的起始位置。fun 返回这个地址，puts(str) 从这里开始输出，结果就是 "efgh"。

- ☐ A. abcdefgh
- ☐ B. abcd
- ☐ C. efgh
- ☐ D. fgh



题目

2-17 分数 2 查看题目详情

对a数组元素不正确的引用是

```
int a []={ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9}, * p=a, i;
```

其中 $0 \leq i \leq 9$ 。

- ☐ A. `a[p-a]`
- ☐ B. `*(&a[i])`
- ☐ C. `p[i]`
- ☐ D. `a[10]`

D。

数组 `a` 有 10 个元素，下标范围是 $0 \sim 9$ ，`a[10]` 访问的是第 11 个元素，已经越界，因此是错误的引用；而 `a[p-a]` 等价于 `a[0]`，`*(&a[i])` 等价于 `a[i]`，`p[i]` 也等价于 `a[i]`，都是合法引用。

下面程序的输出结果是()。

```
# include <stdio.h>
void fun (int *x, int *y)
{
    printf("%d%d", *x, *y);
    *x = 3;
    *y = 4;
}
int main(void)
{
    int x=1, y=2;

    fun(&x, &y);
    printf("%d%d", x, y);

    return 0;
}
```

C。

程序开始时 $x=1, y=2$ ，调用 $\text{fun}(\&x, \&y)$ 后， fun 里第一次 printf 输出当前的 $*x, *y$ 即 12，然后把 $*x$ 改为 3、 $*y$ 改为 4；返回后 x, y 变成 3 和 4，第二次 printf 输出 34，合起来是 1234。

- ☐ A. 2134
- ☐ B. 1212
- ☐ C. 1234
- ☐ D. 2112



13

字符串和 字符串函数

理论课程



福州大学
FUZHOU UNIVERSITY

字符和字符串

- 字符（char类型）

这里仅指ASCII码字符（char类型），现代程序包含汉字、日文、韩文等，则使用short类型，根据编码不同分为（Unicode、GB2312、Big5等码。）

- 可打印字符（0x20-0x7E）

- 字母、数字、运算符号、标点符号和其他符号

- 控制字符（0x00-0x1F, 0x7F）

- 空格、回车、制表符、退格、警报等.....

- 字符串

- 一个有格式的字符数组，以数值为0的元素为结束。

- 物理意义：承载信息的文字

字符串的声明

• 字符串的声明

赋值方法	赋值语句	开辟空间	字符串常量开辟空间	复制数组
以数组形式声明	<code>char c[20];</code>	20B	0B	无初始化
在声明时赋值 (含长度)	<code>char c[20] = "Hello world!";</code>	20B	13B	自c[0]赋为'H'至c[11]赋为'!', c[12]赋为0, 其余赋为0。
在声明时赋值 (不含长度)	<code>char c[] = "Hello world!";</code>	13B	13B	
以指针变量形式赋值	<code>char *p = "Hello world!";</code>	4B/ 8B	13B	将字符串常量的内存首地址赋值给字符指针。

字符串的赋值

• 字符串的赋值

赋值方法	赋值语句
在声明时赋值	<code>char c[] = "Hello world!";</code>
通过字符串复制函数赋值	<code>strcpy(c, "Hello world!");</code>
通过字符串输入函数赋值	<code>scanf("%s" , c);</code>

• 注意事项

- 字符串数组名为常量，仅可在声明语句被字符串常量赋值
- 通过字符串函数赋值前应开辟足够的目标空间

字符串常量的其它形式

- 字符串常量（字符串文字）
 - 一对双引号中包含任何字符，特殊字符转义
 - 字符串允许中间断字（断行）
- 字符串存储时
 - 不包括双引号，包括双引号内的字符，添加结束标志（\0）

```
char greeting[50] = "Hello, and" " how are" " you"  
" today!";
```

```
char greeting[50] = "Hello, and how are you today!";
```

字符串长度函数

strlen

功能	求字符串的长度，即首个空字符的下标。	
格式	<code>size_t strlen(const char* str);</code>	
参数	<code>str</code>	待检查的字符串指针
返回值	成功	字符串的长度，即空字符所在下标
头文件	<code>string.h</code>	
说明	1. 如果该字符串所在空间中不存在空字符，将越界继续寻找空字符。 2. 如果传入空指针，则返回0。	

字符串：长度和数组空间

- 字符串长度：`strlen`函数（应包含`<string.h>`）
 - 从数组起始位置开始数，到第一个数字0的偏移量。
- 字符串空间：`sizeof`操作符
 - 数组声明时，所分配的空间的字节数。

示例程序中为3
 - 字符串常量`sizeof(name)`比`strlen(name)`多1
 - 字符串变量`sizeof(name)`和`strlen(name)`无必然联系

字符形式	W	e	i	\0	*	*	*	越界
数值形式	87	101	105	0	*	*	*	越界
标记	起始			strlen				sizeof
索引	0	1	2	3	4	...	39	40

字符串名称是其指针

- 数组名称的值是其指针（内存首地址） `int a[5];` //a就是 `&a[0]`
 - 字符串名称的值是其指针（内存首地址）
 - 对指针可以间接寻址
- 编译器会在内存中放一段字符：Hello world!\0然后，把这段内存的首地址，当作"Hello world!" 这个表达式的值。

表达式	类型	值
"Hello world!"	字符串，类比数组a	0x3000
*"Hello world!"	字符，类比数组元素*a，即a[0]	72
"Hello world!"[0]	字符，类比数组元素a[0]	72

- 字符串常量为指向字符的指针常量赋值
 - 如果为指针变量赋值，将来修改元素时，产生段错误

`char *p = "Hello world!";` `p[0] = 'h';` `p[0] = 'h';` // 产生段错误

```
/* strcpy.c -- strings as pointers */
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    printf("%s, %p, %c\n", "We", "are", *"space farers");
```

```
    return 0;
```

```
}
```

字符串是数组，其值为字符串首地址p，因此*p为字符串首个元素。

字符串在配合%s或puts输出时输出所有元素（直到0）。

```
We, 00C52010, s
```

字符串数组

- 以指针数组定义字符串数组，每个字符串不定长
 - 声明语句 `char *mytalents[LIM]`
 - 每个数组元素为字符串指针，其值没有必然联系
- 以多维数组定义字符串数组，每个字符串数组定长
 - 声明语句 `char yourtalents[LIM][SLEN]`
 - 每个一级数组元素（如：`yourtalents[1]`）为字符串指针，相互之间距离为SLEN。

```
// arrchar.c -- array of pointers, array of strings
#include <stdio.h>
#define SLEN 40
#define LIM 5
int main(void) {
    const char* mytalents[LIM] = {
        "Adding numbers swiftly",
        "Multiplying accurately", "Stashing data",
        "Following instructions to the letter",
        "Understanding the C language"
    };
    char yourtalents[LIM][SLEN] = {
        "Walking in a straight line",
        "Sleeping", "Watching television",
        "Mailing letters", "Reading email"
    };
    int i;
```



```

puts("Let's compare talents.");
printf("%-36s  %-25s\n", "My Talents", "Your Talents");
for (i = 0; i < LIM; i++)
    printf("%-36s  %-25s\n", mytalents[i], yourtalents[i]);
printf("\nsizeof mytalents: %zd, sizeof yourtalents: %zd\n",
        sizeof(mytalents), sizeof(yourtalents));

return 0;
}

```

Let's compare talents.

My Talents

Adding numbers swiftly

Multiplying accurately

Stashing data

Following instructions to the letter

Understanding the C language

Your Talents

Walking in a straight line

Sleeping

Watching television

Mailing letters

Reading email

sizeof mytalents: 40, sizeof yourtalents: 200

字符串输入

gets

功能	从标准输入流读取字符串，到换行符或文件尾停止。	
格式	<code>char* gets(char* str);</code>	
参数	<i>str</i>	字符串指针，从输入流读取后存入的字符串
返回值	成功	<i>str</i>
	失败	NULL
头文件	<code>string.h</code>	
说明	1. 不进行边界检查，应提前对<i>str</i>开辟足够空间，如果输入超过开辟的空间，会引起段错误（运行错误）。 2. 建议使用<i>fgets()</i>替代<i>gets()</i>，但二者细节处理不同。	

fgets会指定大小，如果超出数组大小，会自动根据定义数组的长度截断。

字符串输出

puts

功能	将字符串附上换行符，写到标准输出流。	
格式	<code>int puts(const char* str);</code>	
参数	<code>str</code>	字符串指针，待写入输出流的字符串
返回值	成功	非负值，没有特别含义。
	失败	EOF
头文件	<code>stdio.h</code>	
说明	1. 使用 <code>puts()</code> 会附上换行符，这点和 <code>fputs()</code> 的处理不同。	

文件按行读

fgets

功能	从文件读取一行	
格式	<code>char *fgets(char *s, int n, FILE *stream);</code>	
参数	<i>s</i>	字符串，读取后字符串存储的位置
	<i>n</i>	接受字符串最大的字符数
	<i>stream</i>	指向待操作文件的指针
返回值	成功	指针 <i>s</i>
	失败	NULL
说明	1. 读入文件填入<i>s</i>，直至回车、第<i>n</i>-1个字符或文件末尾三个条件之一结束，将下一个字符设置为0。 2. 如以回车结束，则加上回车符0x0A（不同于gets()） 3. 读取前触及文件末尾，返回NULL，不改变<i>s</i>的内容	

文件写字符串

fputs

功能	从文件输出一组字符	
格式	<code>int fputs(const char *s, FILE *stream);</code>	
参数	<code>s</code>	读取后，字符串存储的位置
	<code>stream</code>	指向待操作文件的指针
返回值	成功	非负整数（一般为0）
	失败	EOF
说明	1. 输出字符串，不添加回车。（不同于puts()）	

```

/* fgets1.c -- using fgets() and fputs() */
#include <stdio.h>
#define STLEN 14
int main(void)
{
    char words[STLEN];
    puts("Enter a string, please.");
    fgets(words, STLEN, stdin);
    printf("Your string twice (puts(), then fputs()):\n");
    puts(words);
    fputs(words, stdout);
    puts("Enter another string, please.");
    fgets(words, STLEN, stdin);
    printf("Your string I'll\n");
    puts(words);
    fputs(words, stdout);
    puts("Done.");
    return 0;
}

```

```

Your string twice (puts(), then fputs()):
I'll

I'll
Enter another string, please.
go home
Your string twice (puts(), then fputs()):
go home

go home
Done.

```

```

/* fgets3.c  -- using fgets() */
#include <stdio.h>
#define STLEN 10
int main(void)
{
    char words[STLEN];
    int i;
    puts("Enter strings (empty line to quit):");
    while (fgets(words, STLEN, stdin) != NULL && words[0] != '\n')
    { //fgets:从输入流（如键盘）读取一行字符串，最多读取 n-1 个
      字符，并在末尾自动添加空字符 \0。它会保留换行符 \n（如果有的话）。
        i = 0;
        while (words[i] != '\n' && words[i] != '\0')
            i++;
        if (words[i] == '\n')
            words[i] = '\0';
        else // must have words[i] == '\0'

```

```
        while (getchar() != '\n')
            continue;
    puts(words);
}
puts("done");
return 0;
}
```

empty line↵

empty lin

empty lines↵

empty lin

GOOD↵

GOOD

↵

done

字符串函数：字符串格式化

- 从字符串格式化输入：`sscanf(str, fmt, ...)`
 - 功能与`scanf`相同，只是输入为字符串
 - 返回值为正确赋值的参数个数
- 格式化输出到字符串：`sprintf(str, fmt, ...)`
 - 功能与`printf`相同，只是输出为字符串
 - 返回值为被实际赋值的字符串长度

```
char buf[] = "123 4.56 hello";
```

```
int i;
```

```
double d;
```

```
char s[20];
```

```
int n = sscanf(buf, "%d %lf %s", &i, &d, s);
```

```
// i = 123, d = 4.56, s = "hello", n = 3
```

```
char buf[50];
```

```
int a = 3, b = 5;
```

```
int len = sprintf(buf, "%d + %d = %d", a, b, a + b);
```

```
// buf 变成 "3 + 5 = 8", len = 9 (不含结尾的 '\0')
```

字符串函数

- 本节自习：注意输入、输出、作用
 - 长度: `strlen()`
 - 拼接: `strcat()`; `strncat()`
 - 比较: `strcmp()`; `strncmp()`
 - 复制: `strcpy()`; `strncpy()`
 - 打印: `sprintf()`
 - 其它: `strchr()`; `strpbrk()`; `strrchr()`; `strstr()`

ctype.h 字符函数和字符串

- 基于ctype.h字符函数，可用于编写
 - 字符串的转换大写、小写、首字母大写、句首大写、统计空白符个数等函数。
- 使用字符函数前应包含相应文件头
- 由于上述函数较为简单，方便自行编写，不需要特地记忆。

题目

2-18 分数 2 查看题目详情

以下语句或语句组中，能正确进行字符串赋值的是____。

- ☐ A. `char *sp; *sp="right!";`
- ☐ B. `char s[10]; s="right!";`
- ☐ C. `char s[10]; *s="right!";`
- ☐ D. `char *sp="right!";`

D。

"right!" 是字符串常量，必须用指针指向它或用数组在定义时初始化。

- A 中 `sp` 未初始化就用 `*sp` 赋值，非法；
- B 中数组 `s` 已存在，不能再 `=` 直接整体赋字符串；
- C 中 `*s` 是一个 `char`，不能接收字符串地址；
- D 中 `char *sp="right!";` 定义时用指针指向字符串常量，合法。选 D。

题目

2-19 分数 2 查看题目详情

设有下面的程序段：char s[] = "china"; char *p = s; 则下列叙述正确的是_____。

- ☐ A. 数组s中的内容和指针变量p中的内容相等
- ☐ B. s和p完全相同
- ☐ C. s数组长度和p所指向的字符串长度相等
- ☐ D. *p与s[0]相等

D。

s 是字符数组，p 是指向该数组首元素的指针，因此 *p 取得的是第一个字符，正好等于 s[0]；其它选项要么把数组和指针本身混为一谈，要么把数组长度和字符串长度混淆，都是不正确的。

14

结构体与 其它数据格式

理论课程



福州大学
FUZHOU UNIVERSITY

结构体变量的声明

• 格式

声明形式	语句格式	示例
先声明结构体类型，再声明结构体变量	<pre>struct <标签名> { <成员列表> }; struct <标签名> <变量名列表>;</pre>	<pre>struct book { char title[MAXTITL]; float value; }; struct book mybook;</pre>
不声明结构体类型，只声明结构体变量	<pre>struct { <成员列表> } <变量名列表>;</pre>	<pre>struct { char title[MAXTITL]; float value; } mybook;</pre>
声明结构体类型和变量	<pre>struct <标签名> { <成员列表> } <变量名列表>;</pre>	<pre>struct book { char title[MAXTITL]; float value; } mybook;</pre>

结构体变量的初始化

- 结构体变量应先声明后使用，先写后读
- 类似于一般变量和数组

```
int count = 0;  
int fibo[7] = { 0,1,1,2,3,5,8 };
```

- 结构体可以用复合文字初始化

```
struct book library = {  
    "The Pious Pirate and the Devious Damsel",  
    "Renee Vivotte",  
    1.95  
};
```

– 注意：只能在声明时初始化，各个初始值的类型要对应。

结构体内变量的访问

- 结构体就像一个“超级数组”

- 格式: `[tag].[member]`

- 示例: `bill.title`

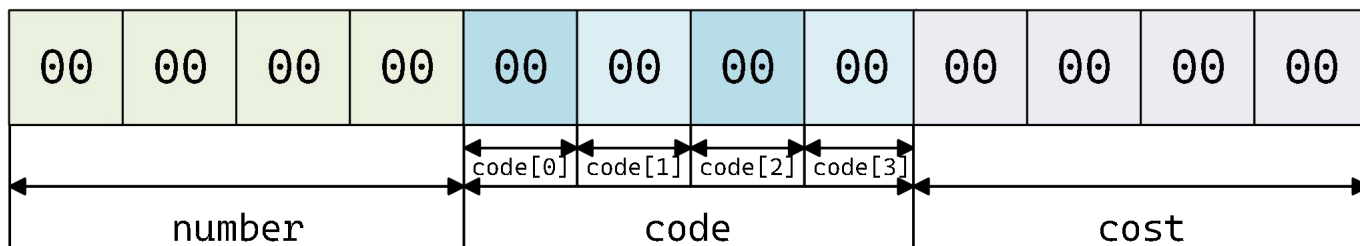
- 结构体指定初始化项目

```
struct book surprise = { .value = 10.99 };  
struct book gift = { .value = 25.99,  
    .author = "James Broadfool",  
    .title = "Rue for the Toad" };
```

结构体变量的存储

- 结构体依其成员顺序在内存中存储
 - 便于在解析格式化的对象时指针类型的强制类型转换
 - 例如：IP报文的头部，文件头

```
struct stuff {  
    int number;  
    char code[4];  
    float cost;  
};
```



```

/* book.c -- one-book inventory */
#include <stdio.h>
#include <string.h>
char * s_gets(char * st, int n);
#define MAXTITL  41    /* maximum length of title + 1          */
#define MAXAUTL  31    /* maximum length of author's name + 1 */
struct book {          /* structure template: tag is book      */
    char title[MAXTITL];
    char author[MAXAUTL];
    float value;
};                      /* end of structure template          */
int main(void) {
    struct book library; /* declare library as a book variable */
    printf("Please enter the book title.\n");
    s_gets(library.title, MAXTITL); /* access to the title portion */
    printf("Now enter the author.\n");
    s_gets(library.author, MAXAUTL);
    printf("Now enter the value.\n");
    scanf("%f", &library.value);
    printf("%s by %s: $%.2f\n", library.title, library.author,
library.value);
}

```

```

printf("%s: \"%s\" ($%.2f)\n", library.author,
      library.title, library.value);
printf("Done.\n");
return 0;
}
char * s_gets(char * st, int n) {
    char * ret_val;
    char * find;
    ret_val = fgets(st, n, stdin);
    if (ret_val) {
        find = strchr(st, '\n'); // look for newline
        if (find)                // if the address is not NULL,
            *find = '\0';        // place a null character
there
    }
    else
        while (getchar() != '\n')
            continue;
    return ret_val;
}

```

Please enter the book title.

C Primer Plus↓

Now enter the author.

Stephen Prata↓

Now enter the value.

60.00↓

C Primer Plus by Stephen Prata: \$60.00
 Stephen Prata: "C Primer Plus" (\$60.00)
 Done.

结构体数组

- 结构体数组的声明 `struct <标签名> <数组名>[<数组大小>];`
- 结构体数组成员的访问 `<数组名>[<数组下标>].<成员>;`

表达式	说明
<code>library</code>	结构体数组
<code>library[2]</code>	<code>library</code> 第3个元素，结构体
<code>library[2].title</code>	<code>library[2]</code> 的title属性，字符串
<code>library[2].title[4]</code>	<code>library[2].title</code> 的第5个字符

- 结构体在内存的存储
 - 数组元素按结构体属性次序
 - 数组按元素次序

	char title[40]	char author[40]	float value
library[0]	library[0].title	library[0].author	library[0].value
library[1]	library[1].title	library[1].author	library[1].value
library[2]	library[2].title	library[2].author	library[2].value
...	...	...	...
library[99]	library[99].title	library[99].author	library[99].value

declaration: `struct book library[MAXBKS]`

指向结构体的指针

- 结构体指针的声明、查址、取址与一般类型一致

```
struct guy* him;
```

```
him = &barney;  
him = &fellow[0];
```

- 指针成员运算符 -> <指针名>-><成员名>

– 以下三种写法等价（应使用第一种写法）

```
him->income
```

```
(*him).income
```

```
him[0].income
```

指向结构体的指针

- 结构体指针的声明

```
struct <标签名> * <指针名>;
```

- 结构体指针成员的访问

```
<指针名>-><成员名>
```

表达式	说明
him->income	指针形式，指针时推荐
him[0].income	结构体数组形式，数组时推荐
(*him).income	结构体形式，不推荐

```

/* friends.c -- uses pointer to a structure */
#include <stdio.h>
#define LEN 20
struct names {
    char first[LEN];
    char last[LEN];
};
struct guy {
    struct names handle;
    char favfood[LEN];
    char job[LEN];
    float income;
};
int main(void) {
    struct guy fellow[2] = {
        {{ "Ewen", "Villard"}, "grilled salmon", "personality coach",
        68112.00
        },
        {{ "Rodney", "Swillbelly"}, "tripe", "tabloid editor",
        432400.00
        }
    };
};

```



```

    struct guy* him; /* here is a pointer to a structure */
    printf("address #1: %p #2: %p\n", &fellow[0],
&fellow[1]);
    him = &fellow[0]; /* tell the pointer where to point */
    printf("pointer #1: %p #2: %p\n", him, him + 1);
    printf("him->income is $%.2f: (*him).income is $%.2f\n",
        him->income, (*him).income);
    him++; /* point to the next structure */
    printf("him->favfood is %s: him->handle.last is %s\n",
        him->favfood, him->handle.last);
    return 0;
}

```

```

address #1: 00E1FC9C #2: 00E1FCF0
pointer #1: 00E1FC9C #2: 00E1FCF0
him->income is $68112.00: (*him).income is $68112.00
him->favfood is tripe: him->handle.last is Swillbelly

```

结构体复合文字

- 数组有复合文字，结构体同样也有

```
(struct book) {"Mr. Bouncy's Nice Hat",  
               "Fred Winsome",  
               5.99};
```

- 伸缩型数组成员

– 一定是数组的最后一个成员，并且有其他成员

```
struct flex {  
    size_t count;  
    double average;  
    double scores[];    // flexible array member  
};
```

```
pf1 = malloc(sizeof(struct flex) + n * sizeof(double));
```

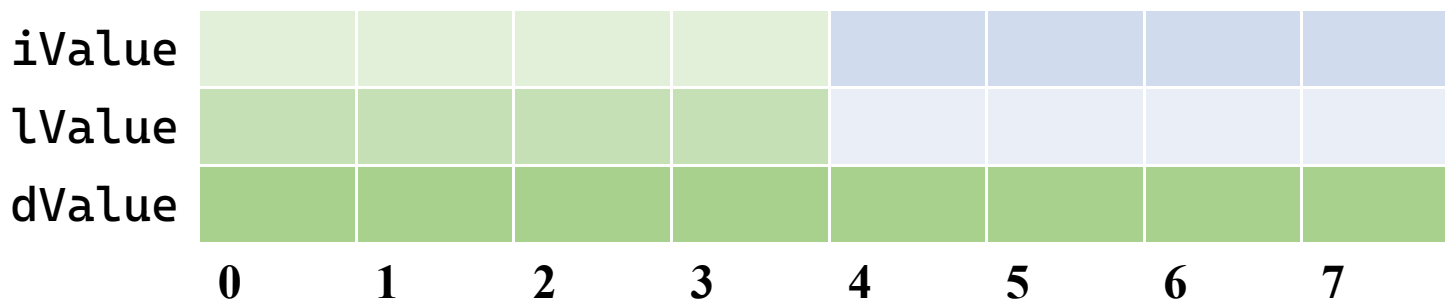
在函数中访问结构体信息

- 传递结构体成员 `void showinfo(const char* info);`
 - 优点：函数与结构体无关，适用范围广
- 传递结构体地址 `void showinfo(const struct namect* p);`
 - 优点：在传递大型结构体时，指针传递只需赋值4或8字节
 - 为了防止修改可加const
- 传递结构体整体 `void showinfo(struct namect info);`
 - 缺点：传递大型结构体需要占用更多的内存空间
 - 仅用于较小的结构体

联合体

- 联合体（**union**）是一个能在同一个存储空间（但不同时）存储不同类型数据的数据类型。

```
union NumericType
{
    int      iValue;
    long     lValue;
    double   dValue;
};
```



联合体的使用

• 声明

```
union hold fit;           // union variable of hold type
union hold save[10];      // array of 10 union variables
union hold* pu;           // pointer to a variable of hold type
```

• 初始化

```
union hold valA;
valA.letter = 'R';
union hold valB = valA;    // initialize one union to another
union hold valC = { 88 };  // initialize digit member of union
union hold valD = { .bigfl = 118.2 }; // designated initializer
```

• 一般用法

```
fit.digit = 23;           // 23 is stored in fit; 2 bytes used
fit.bigfl = 2.0;          // 23 cleared, 2.0 stored; 8 bytes used
fit.letter = 'h';         // 2.0 cleared, h stored; 1 byte used
```

联合体的使用

- 指针用法

```
pu = &fit;  
x = pu->digit; // same as x = fit.digit
```

- 错误用法：应避免使用非初始化的内存区域

```
fit.letter = 'A';  
flnum = 3.02 * fit.bigfl; // ERROR ERROR ERROR
```

- 结构体可以和联合体搭配使用

```
union data {  
    struct owner owncar;  
    struct leasecompany  
leasecar;  
};
```

```
struct car_data {  
    char make[15];  
    int status; /* 0 owned, 1 leased */  
    union data ownerinfo;  
    ...  
};
```

枚举

- 自定义一种类型并且自定义其值

```
enum spectrum { red, orange, yellow, green, blue, violet };  
enum spectrum color;
```

- 使用方法

```
color = blue;  
if (color == yellow)  
    ...;  
for (color = red; color <= violet; color++)  
    ...;
```

- 枚举常量

- 如果有赋值则用赋值；如果无赋值，则为上一个值加一；
首个元素默认为0

```
enum kids { nippy, slats, skippy, nina, liz };  
enum feline { cat, lynx = 10, puma, tiger };
```

```

/* enum.c -- uses enumerated values */
#include <stdio.h>
#include <string.h>    // for strcmp(), strchr()
#include <stdbool.h>   // C99 feature
char* s_gets(char* st, int n);
enum spectrum { red, orange, yellow, green, blue, violet };
const char* colors[] = { "red", "orange", "yellow",
    "green", "blue", "violet" };
#define LEN 30
int main(void) {
    char choice[LEN];
    enum spectrum color;
    bool color_is_found = false;
    puts("Enter a color (empty line to quit):");
    while (s_gets(choice, LEN) != NULL && choice[0] != '\0') {
        for (color = red; color <= violet; color++) {
            if (strcmp(choice, colors[color]) == 0) {
                color_is_found = true;
                break;
            }
        }
    }
}

```



```

if (color_is_found)
    switch (color) {
        case red: puts("Roses are red.");
                    break;
        case orange: puts("Poppies are orange.");
                    break;
        case yellow: puts("Sunflowers are yellow.");
                    break;
        case green: puts("Grass is green.");
                    break;
        case blue: puts("Bluebells are blue.");
                    break;
        case violet: puts("Violets are violet.");
                    break;
    }
else
    printf("I don't know about the color %s.\n", choice);
color_is_found = false;
puts("Next color, please (empty line to quit):");
}
puts("Goodbye!");
return 0;
}

```

```

char* s_gets(char* st, int n) {
    char* ret_val;
    char* find;
    ret_val = fgets(st, n, stdin);
    if (ret_val) {
        find = strchr(st, '\n'); // look for newline
        if (find)                // if the address is not NULL,
            *find = '\0';        // place a null character there
        else
            while (getchar() != '\n')
                continue;        // dispose of rest of line
    }
    return ret_val;
}

```

Enter a color (empty line to quit):

Red↓

I don't know about the color Red.

Next color, please (empty line to quit):

blue↓

Bluebells are blue.

Next color, please (empty line to quit):

pink↓

I don't know about the color pink.

Next color, please (empty line to quit):

Goodbye!

共享命名空间

- C使用命名空间识别一个名字的程序部分
- 作用域
- 结构体、联合体、枚举标记共享一个命名空间
 - 与普通变量不冲突

```
struct rect { double x; double y; };  
int rect; // not a conflict in C
```

类型定义简介

- 类型定义

- 格式: `typedef <基本类型> <带修饰符的类型>;`
- 示例: 定义无符号字符为BYTE（字节）类型

```
typedef unsigned char BYTE;  
BYTE x, y[10], * z;
```

- 作用: 自定义类型名

- 自定义结构体
- 自定义过于复杂的类型

- 注意: 仅限针对类型, 而非对值的定义

类型定义使用范例

- 自定义结构体

定义结构体名和类型名	定义匿名结构体和类型名
<pre>typedef struct complex { float real; float imag; } COMPLEX;</pre>	<pre>typedef struct { float real; float imag; } COMPLEX;</pre>

- 自定义过于复杂的类型

```
typedef char(*FRPTC())[5];
```

1. FRPTC(): 表示一个函数
2. *FRPTC(): 这个函数返回一个指针
3. (*FRPTC())[5]: 这个指针指向含 5 个 char 的数组
4. 前面的 char 说明数组元素类型是 char

指向函数的指针

- 函数的名称是函数所在代码段的首地址（类似数组）
- 作用：函数指针可以作为另一个函数的参数
- 定义与赋值

```
void ToUpper(char*);  
void ToLower(char*);  
int round(double);  
void (*pf)(char*);  
pf = ToUpper; // valid, ToUpper is address of the function  
pf = ToLower; // valid, ToLower is address of the function  
pf = round; // invalid, round is the wrong type of function  
pf = ToLower(); // invalid, ToLower() is not an address
```

使用指向函数的指针

- 使用指针指向的函数甚至指针本身来使用函数指针
 - 因为函数名本身就是一个指针

```
void ToUpper(char*);  
void ToLower(char*);  
void (*pf)(char*);  
char mis[] = "Nina Metier";  
pf = ToUpper;  
(*pf)(mis);    // apply ToUpper to mis (syntax 1)  
pf = ToLower;  
pf(mis);        // apply ToLower to mis (syntax 2)
```

题目

2-1 分数 2 查看题目详情

对于以下结构定义，`++p->str` 中的 `++` 加在___。

```
struct {  
    int len;  
    char *str;  
} *p;
```

- ☐ A. 指针str上
- ☐ B. 指针p上
- ☐ C. str指的内容上
- ☐ D. 以上均不是

A。

->的优先级高于前置 ++，`++p->str` 等价于 `++(p->str)`，也就是让结构体成员指针 str 自身加 1，而不是让指针 p 或 *str 指向的内容加 1，因此选A“指针 str 上”。

题目

2-5 分数 2 查看题目详情

设有如下定义，则对data中的a成员的正确引用是（）。

```
struct sk{ int a; float b; } data, *p=&data;
```

- ☐ A. (*p).data.a
- ☐ B. (*p).a
- ☐ C. p->data.a
- ☐ D. p.data.a

B。

因为 p 是指向结构体变量 data 的指针，要通过指针访问结构体成员 a，写法是 (*p).a（或等价的 p->a）。

选项 A 和 C 里多写了一个不存在的成员 data，结构体里只有 a 和 b；

选项 D 用的是 p.data.a，p 是指针不是结构体变量，不能用点运算符。

所以只有 B (*p).a 正确。

题目

2-9 分数 2 查看题目详情

若有下列定义，则以下不合法的表达式是（ ）。

```
struct student{
    int num;
    int age;
};
struct student stu[3] = {{101, 20}, {102, 19}, {103, 20}}, *p = stu;
```

- ☐ A. (p++)->num
- ☐ B. p++
- ☐ C. (*p).num
- ☐ D. p = &stu.age

D。

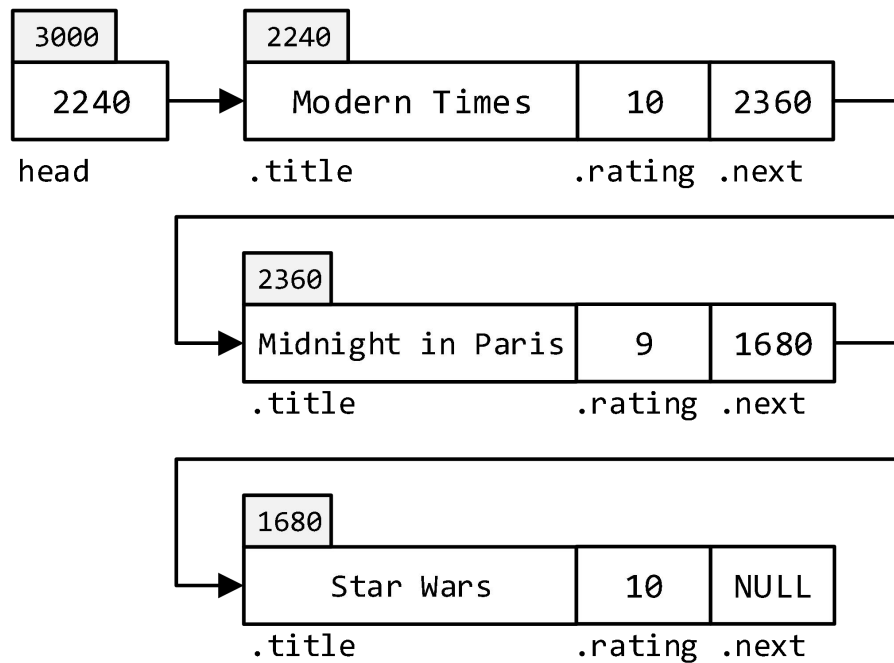
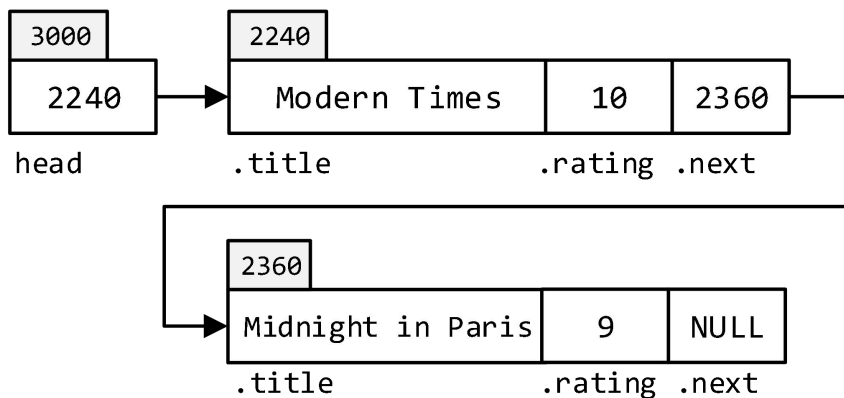
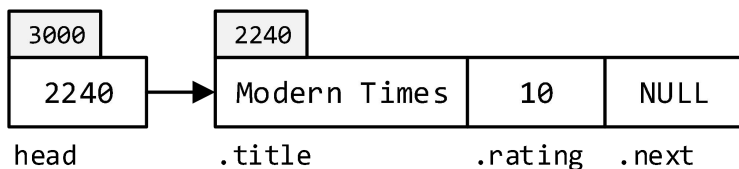
stu 是结构体数组，不是单个结构体变量，不能写成 stu.age；而且 p 的类型是 struct student *，&stu.age 即使存在也会是指向 int 的指针，类型也不匹配，所以只有 D 是不合法的表达式。

高级数据表示



从数组到链表

- 理想情况下，你希望不确定地添加数据
 - 依次输入多部电影，输入结束前总数未知
- 方案



数据结构

- 结点

- 数据（**data**是数据域的示例）

- 下一结点指针 `p->next = NULL;`

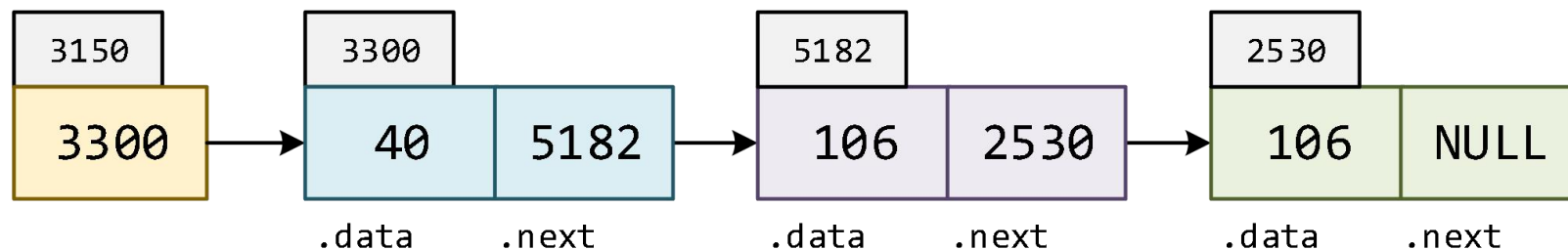
```
typedef struct node {  
    int data;  
    struct node * next;  
} node, * linklist;
```

- 链表

- 链表头：指向第一个结点的指针

- 链表中：**next**成员指向下一个结点

- 链表尾：**next**成员置为空



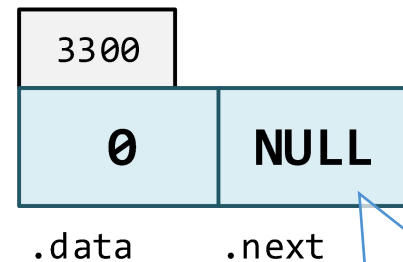
创建结点并链接到另一个结点

- 新建结点

```
node *p = (node*)malloc(sizeof(node));
```

- 赋初始值

- next初始值一般赋值为NULL
- 成员初始值一般按惯例赋值



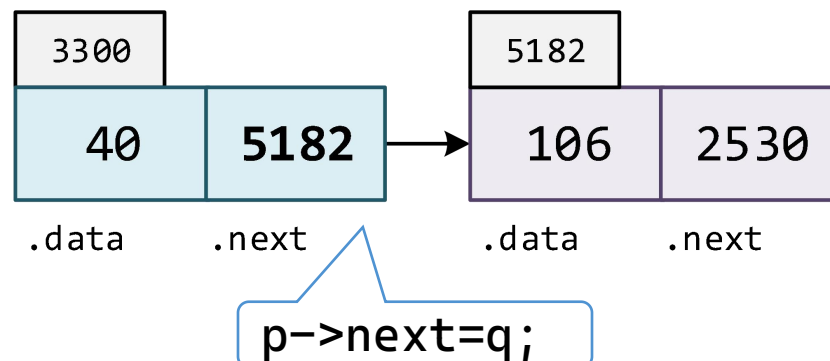
```
p->next=NULL;
```

```
p->data = 0;  
p->next = NULL;
```

- 形成前后顺序

- next赋值为另一结点的地址

```
p->data = 40;  
p->next = q;
```



创建结点并链接到另一个结点

- 完整程序

```
node* create_node() {  
    node* current = (node*)malloc(sizeof(node));  
    if (current == NULL) {  
        fprintf(stderr, "%s\n", "Failed to allocate the  
new node due to not enough memory space.");  
        return NULL;  
    }  
    current->data = 0;  
    current->next = NULL;  
    return current;  
}
```

创建列表：尾插法

• 示例代码

```
while (get_data_from_input(&data) == INPUT_VALID) {  
    current = create_node();  
    if (current == NULL)  
        break;  
    current->data = data;  
    if (previous == NULL)  
        list = current;  
    else  
        previous->next = current;  
    previous = current;  
}
```

第一层：结点相关操作

- 1、分配空间
- 2、内存耗尽处理
- 3、赋初值

第二层：结点链接

- 1、list为空时，当前结点为头结点；
- 2、list不为空时，上一结点的后续指向当前结点；
- 3、记录当前结点为下次循环的前结点

创建列表：头插法

• 示例代码

```
linklist creat_from_head() {  
    linklist list = NULL;  
    node* current = NULL;  
    int data;  
    while (get_data_from_input(&data) == INPUT_VALID) {  
        current = create_node();  
        if (current == NULL)  
            break;  
        current->data = data;  
        current->next = list;  
        list = current;  
    }  
    return list;  
}
```

第一层：结点相关操作

- 1、分配空间
- 2、内存耗尽处理
- 3、赋初值

第二层：结点链接

- 1、当前结点的下一结点是原链表头；
- 2、新链表头应为新建的结点；

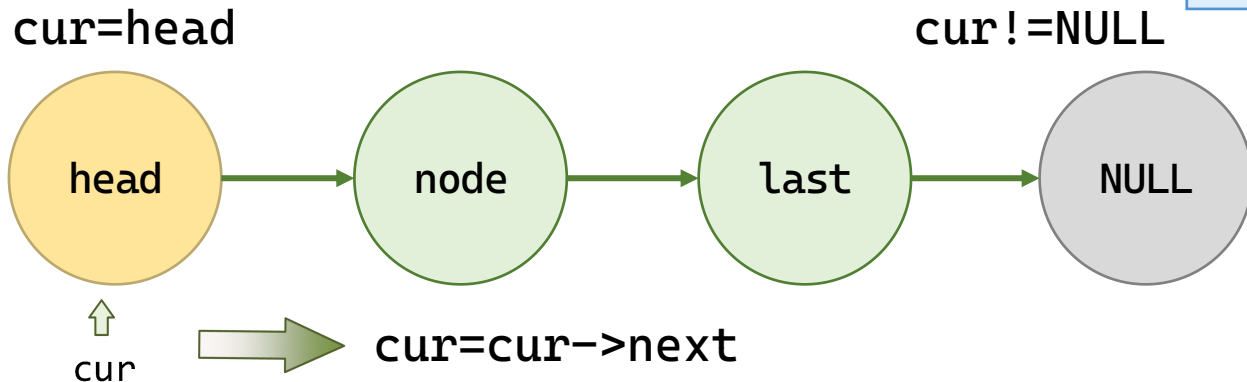
遍历链表

- 按顺序遍历数组和链表比较

功能	数组	链表
定位到第一个元素	$i=0$	$cur=head$
判断最后一个元素	$i < N$	$cur \neq NULL$
移动到下一个元素	$i++$	$cur=cur \rightarrow next$

```
i = 0;  
while (i < N) {  
    ...;  
    i++;  
}
```

```
cur = head;  
while (cur != NULL) {  
    ...;  
    cur = cur->next;  
}
```



计算元素个数

- 示例代码

```
size_t count(const linklist list) {  
    size_t i;  
    node* current = list;  
    for (i = 0; current; ++i)  
        current = current->next;  
    return i;  
}
```

计数

```
cur = head;  
while (cur != NULL) {  
    ++i;  
    cur = cur->next;  
}
```

定位元素

• 示例代码

```
node* get_item(const linklist list, const size_t offset) {  
    size_t i;  
    if (list == NULL) {  
        fputs("Empty list.\n", stderr);  
        return NULL;  
    }  
    node* current = list;  
    for (i = 0; i < offset && current; i++)  
        current = current->next;  
    return current;  
}
```

如果没找到，则current为NULL
不需要处理

查找元素

• 示例代码

```
node* locate(const linklist list, const int data, size_t* offset) {  
    size_t i;  
    if (list == NULL) {  
        fputs("Empty list.\n", stderr);  
        return NULL;  
    }  
    node* current = list;  
    for (i = 0; current && current->data != data; ++i)  
        current = current->next;  
    if (current)  
        *offset = i;  
    return current;  
}
```

先有cur不为空才有cur->data

如果未找到不应有副作用

显示链表

• 示例代码

```
void show(const linklist list) {
    node* current = list;
    size_t i = 0;
    if (current == NULL) {
        fputs("Empty list.\n", stderr);
        return;
    }
    printf("%-6s\t%-8s\t%-16s\t%-16s\n", "ID", "Data", "Next", "Address");
    printf("=====\t=====\t=====\t=====\n");
    while (current != NULL) {
        printf("%4zu \t%6d \t%016p\t%016p\n", ++i, current->data, current->next, current);
        current = current->next;
    }
}
```

插入元素

• 示例代码

```
int insert_item(linklist* list, const size_t offset, const int data) {  
    node* previous, * current;  
    size_t i;  
    previous = *list;  
    if (offset == 0 || *list == NULL) {  
        current = create_node();  
        if (current == NULL)  
            return ERROR;  
        current->data = data;  
        current->next = *list;  
        *list = current;  
        return OK;  
    }  
}
```

在头插入的情况：
第一层，新建结点和初始化。

第二层：结点链接（参考头插法）
1、当前结点的下一结点是原链表头；
2、新链表头应为新建的结点；

插入元素

• 示例代码

```
for (i = 0; i < offset - 1 && previous->next; ++i)
    previous = previous->next;
if (i != offset - 1) {
    fputs("Failed to find the offset.", stderr);
    return ERROR;
}
current = create_node();
if (current == NULL)
    return ERROR;
current->data = data;
current->next = previous->next;
previous->next = current;
return OK;
}
```

寻找插入位置的前一结点
如果不存在该位置报错

一般插入的情况：
第一层，新建结点和初始化。

第二层：结点链接
1、新建结点的后续是前一结点的下一结点；
2、前一结点的后续是新建结点；

删除元素

• 示例代码

```
int remove_item(linklist* list, const size_t offset, int* data) {  
    node* previous = *list, * current = NULL;  
    size_t i;  
    if (*list == NULL) {  
        fputs("Empty list.", stderr);  
        return ERROR;  
    }  
    if (offset == 0) {  
        *data = previous->data;  
        *list = previous->next;  
        free(previous);  
        return OK;  
    }  
}
```

在头删除的情况：
调整结点链接，当前链表头后移；
释放原链表头。

删除元素

• 示例代码

```
for (i = 0; i < offset - 1 && previous->next; ++i)
    previous = previous->next;
current = previous->next;
if (current == NULL) {
    fputs("Failed to find the offset.", stderr);
    return ERROR;
}
*data = current->data;
previous->next = current->next;
free(current);
return OK;
}
```

寻找删除位置的前一结点
如果不存在该位置报错

一般删除的情况：
调整结点链接，前一结点跳过待删结点，后续为待删结点的后续；
释放待删结点。

链表的销毁

• 示例代码

```
node* destroy(linklist* list) {  
    node* current = *list, * next;  
    while (current != NULL) {  
        next = current->next;  
        free(current);  
        current = next;  
    }  
    *list = NULL;  
    return NULL;  
}
```

```
while (current != NULL) {  
    free(current);  
    current = current->next;  
}
```

遍历链表

如果先释放cur，则cur->next无效，因此应先暂存该值，再释放

重置链表头为空

题目

2-10 分数 2 查看题目详情

以下结构类型可用来构造链表的是 () 。

- ☐ A. `struct aa{ int a; int * b; };`
- ☐ B. `struct bb{ int a; bb * b; };`
- ☐ C. `struct cc{ int * a; cc b; };`
- ☐ D. `struct dd{ int * a; aa b; };`

B。

构造链表时，每个节点里要包含一个“指向本结构体类型的指针”作为 next 指针。只有 B 中的 `bb *b;`（指向同类型 `bb` 的指针）满足这个条件；A 只是 `int *`，C 含有自身类型的普通成员会导致无法确定大小，D 里指针和结构体类型也不对应，因此只有 B 能用来构造链表。

题目

2-11 分数 2 查看题目详情

在一个单链表head中，若要在指针p所指结点后插入一个q指针所指结点，则执行（）。

- ☐ A. $p \rightarrow next = q \rightarrow next; q \rightarrow next = p;$
- ☐ B. $q \rightarrow next = p \rightarrow next; p = q;$
- ☐ C. $p \rightarrow next = q \rightarrow next; p \rightarrow next = q;$
- ☐ D. $q \rightarrow next = p \rightarrow next; p \rightarrow next = q;$

D。

在单链表中要把结点 q 插到结点 p 后面，必须先让 $q \rightarrow next = p \rightarrow next$ 把 q 接到原来 p 的后继上，再让 $p \rightarrow next = q$ 把 p 指向 q，即 $q \rightarrow next = p \rightarrow next; p \rightarrow next = q;$ ，也就是选项 D，其他写法要么断链要么没把 q 接到 p 后面。

题目

2-14 分数 2 查看题目详情

以下程序段的功能是：输入一行字符，按输入的逆序建立一个链表。

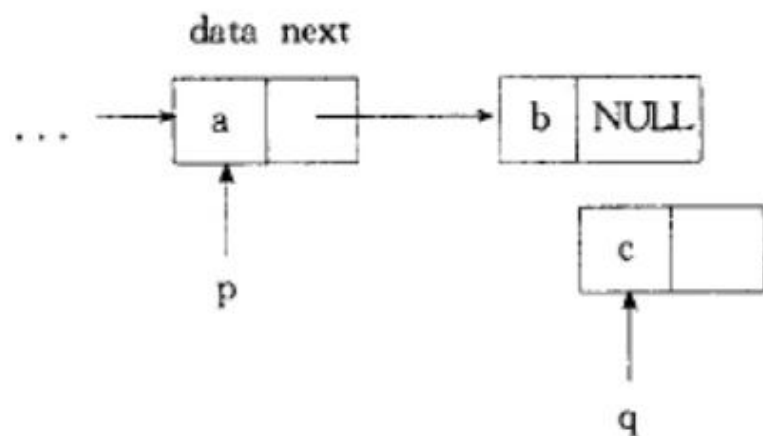
```
char c;  
struct node{  
    char info;  
    struct node *link;  
} *top, *p;  
  
top = NULL;  
while ( (c = getchar() ) != '\n'){  
    p = (struct node*) malloc(sizeof(struct node));  
    p->info = c;  
    _____ ;  
    top = p;  
}
```

- ☐ A. top->link = p
- ☐ B. p->link = top
- ☐ C. top = p->link
- ☐ D. p = top->link

B。

本题要把新结点 **p** 插到链表头部，使链表按输入字符的“逆序”建立，所以新结点的 link 应该指向原来的表头 **top**，即 **p->link = top;**，再令 **top = p;** 把新结点变成新的表头。因此应选 **B**，其它选项不是头插法的正确链接方式，且 A 在 **top** 为 **NULL** 时还会非法访问。

若已建立下面的链表结构，指针 p 、 q 分别指向图中所示结点，则不能将 q 所指结点插入到链表末尾的语句是（ ）。



- ☐ A.
- ```
q->next = NULL;
p = p->next;
p->next = q;
```
- ☐ B.
- ```
p = p->next;
q->next = p->next;
p->next = q;
```
- ☐ C.
- ```
p = p->next;
q->next = p;
p->next = q;
```
- ☐ D.
- ```
p = (*p).next;
(*q).next = (*p).next;
(*p).next = q;
```

C。
选 C 因为它执行 $q \rightarrow next = p$; $p \rightarrow next = q$ ，使得结点 b 和结点 q 互相指向，形成循环链（ $b \rightarrow q \rightarrow b \dots$ ），链表尾结点不再以 NULL 结束，因此不能把 q 正确插入到链表末尾。

谢谢观看

